

programs with only simple variables and fixed-size arrays,  $j$  can be determined at compile time. The offset  $j$  starts with 0 for the first variable. Block level  $B$  is known by the compiler, so the TA of every variable can be fixed at compile time.

For example, consider the program

```
begin integer A;
  procedure P;
    begin integer X,Y;
      procedure Q;
        begin integer Z;
          Z := X;
          X := X + A;
          Y := X + Z
        end;
        X := A;
        A := A + 1;
        Q
      end;
      A := 2;
      P;
      begin integer C;
        C := A;
        P;
        print(C)
      end
    end
  end
```

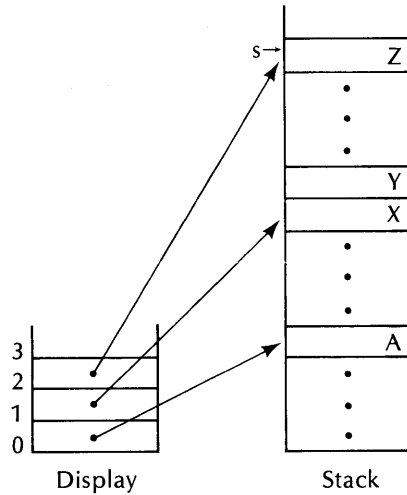
Then the TA's of the variables are as follows:

|   |       |                       |
|---|-------|-----------------------|
| A | [0,0] | e.g., block 0, word 0 |
| X | [1,0] | e.g., block 1, word 0 |
| Y | [1,1] | e.g., block 1, word 1 |
| Z | [2,0] |                       |
| C | [1,0] | Note: Same as X       |

TA's will be used, in conjunction with the display memory, to find the locations of variables at run-time. The rule is relatively simple:

The address of the proper activation of a variable  $X$  with  $TA = [B,j]$  is  $D(B)+j$ .

Recall that  $D(B)$  is the contents of the  $B$ 'th display register. The display register will be used to keep track of the locations of the various variable



**Figure 9.3.** Stack containing (among other information) four local variables (A, X, Y, and Z), and the display pointers. On this run-time configuration, a procedure at block level 2 is being executed.

domains in the stack. Those variables in block B will be in the stack relative to  $D(B)$ .

For example, while the statement  $Z := X$  in the above program is being executed, the display and stack will appear as in figure 9.3. This scheme clearly permits other variables to be stored in the spaces between  $D(0)$ ,  $D(1)$ , and  $D(2)$ .

We now redefine the addressing instructions as follows:

```

LV B,j:  Load value
          $s \leftarrow s + 1$ 
          $C(s) \leftarrow C(D(B) + j)$ 

LA B,j:  Load address
          $s \leftarrow s + 1$ 
          $C(s) \leftarrow D(B) + j$ 

STD B,j: Store direct
          $C(D(B) + j) \leftarrow C(s)$ 
          $s \leftarrow s - 1$ 

```

### 9.6.3. Block Entry and Exit

We need another change in our implementation. Rather than stacking the old values of variables when a block is entered (as before), we reserve space in

the stack for the new activations of variables whenever a block is entered. We need two new operations for this, BE and EB. These operations will have to be expanded later, but for now:

```

BE  B,n:  Block entry at level B, with n variables
          D(B) ← s + 1    set up display element
          s ← s + n        reserve n words

EB  n:     Exit block with n variables
          s ← s - n        release n words

```

The compiler will always know the appropriate B and n values to use for these instructions. Also note that the display-top variable b is not involved.

BE can be used instead of INCS n for most purposes. In Algol, arrays can only be declared upon entering a block. If they have fixed dimensions, then the total space needed for them and the simple variables can be worked out by the compiler. All the space may then be allocated by a single BE instruction.

Arrays with variable dimensions (dimension determined at run-time) must be allocated dynamically on the stack or in the heap through special AOC instructions. However, a dope vector of fixed length can be allocated by the compiler. A complete discussion of dynamic allocation is given later.

#### Example.

The translation of the above program is then:

```

          BE 0,1 begin integer A {block 0, 1 word}
          BR STRT {branch around procedure code}
              procedure P
P          BE 1,2 begin integer X,Y {block 1, 2 words}
          BR PSTR {branch around procedure Q code}
Q          BE 2,1 procedure Q begin integer z
          LV 1,0 Z := X
          STD 2,0
          LV 1,0 X := X + A
          LV 0,0
          ADD
          STD 1,0
          LV 1,0 Y := X + Z
          LV 2,0
          ADD
          STD 1,1
          EB 1 end
          RTN

```

```

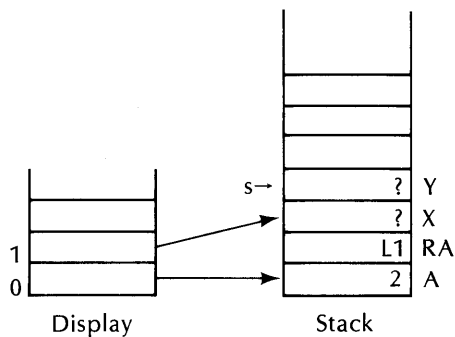
PSTR  LV  0,0  X := A {start of procedure P code}
      STD 1,0
      LV  0,0  A := A+1
      LC  1
      ADD
      STD 0,0
      CP  Q    Q
L3    EB  2    end
      RTN
STRT  LC  2    A := 2 {start of outer block program}
      STD 0,0
      CP  P    P
L1    BE  1,1  begin integer C
      LV  0,0  C := A
      STD 1,0
      CP  P    P
L2    LV  1,0  print(C)
      PRINT
      EB  1    end
      EB  1    end
      HALT

```

This program will show a flaw in our design. Let us simulate it. After the first call of P and execution of its BE, the display and stack appear as in figure 9.4.

After the call of Q and its BE operation, the display and stack appear as in figure 9.5. Immediately after the Q return, the display and stack appear as in figure 9.6.

The return from P just moves the s-pointer to A's location again. After the block for C, P, and Q have been entered again, the display and stack appear as



**Figure 9.4.** A configuration during procedure execution (see text).

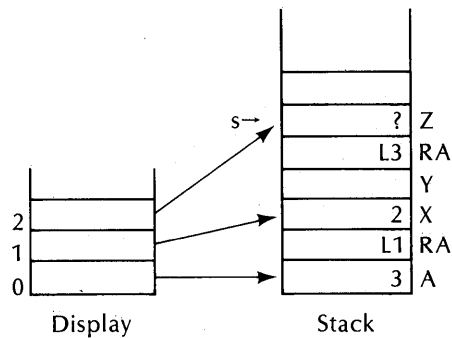


Figure 9.5. A configuration during procedure execution (see text).

in figure 9.7. Notice that there is no display entry pointing to C's location, as is required by TRR from the environment of Q. Instead, the environment seems to be that of the block containing Q's call, namely variables A, X, and Y. Of course, C cannot be accessed from within P, however, it should be accessible from within Q, and it isn't.

Now Q stores A into what it thinks is C. However, the display is such that A is actually stored into X. After procedures P and Q have run to completion and returned, the instructions

```
LV 1,0
PRINT
```

will print the value of X, not C, as they were supposed to. The stack configuration after exiting from Q and P is shown in figure 9.8. Register b is 0 and most of the stack has been dropped.

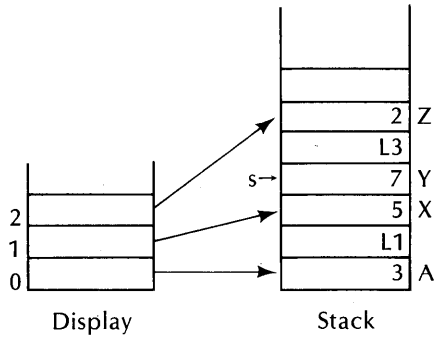


Figure 9.6. A configuration during procedure execution (see text).

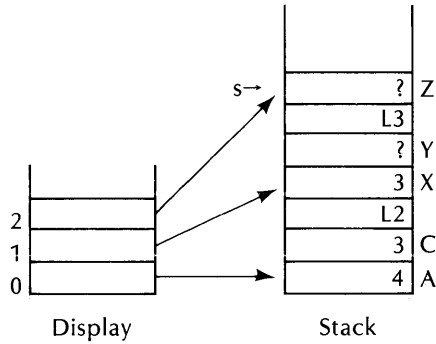


Figure 9.7. A configuration during procedure execution (see text).

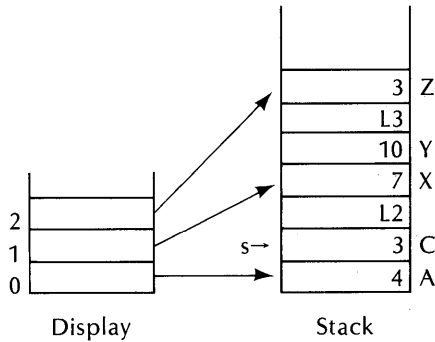


Figure 9.8. A configuration during procedure execution (see text).

The essential problem is that the display entries of a block are lost if a call at block level  $m$  is to a procedure at block level  $n$ , where  $n < m$ . Such a call is called an *up-level call*. We clearly must set the display correctly on an up-level call to correspond to the environment of the called procedure ( $BL = n$ ), then reset it (to  $BL = m$ ) upon exit.

#### 9.6.4. The Static Display Chain

We shall keep a complete copy of the current display in the stack, as well as previous ones which will be needed later, tucked away among the variables. These copies are called *static chains* because they mirror the display, but are not actively involved in addressing variables.

We shall also need register  $b$ , which always contains the  $BL$  of the block whose code is executing;  $b$  also serves as a stack-top index for the display.

We shall arrange that the following is always true, for  $i = 1, 2, \dots, b$ :

$$C(D(i) - 1) = D(i - 1)$$

This relation is shown in figure 9.9 for  $b=2$ . Note that the top display element  $D(b)$  is not in the stack and that the element  $i$  is just below the location pointed to by  $D(i-1)$ . Also,  $C(D(0)-1)$  can contain anything—this word is never accessed by the display manipulation system.

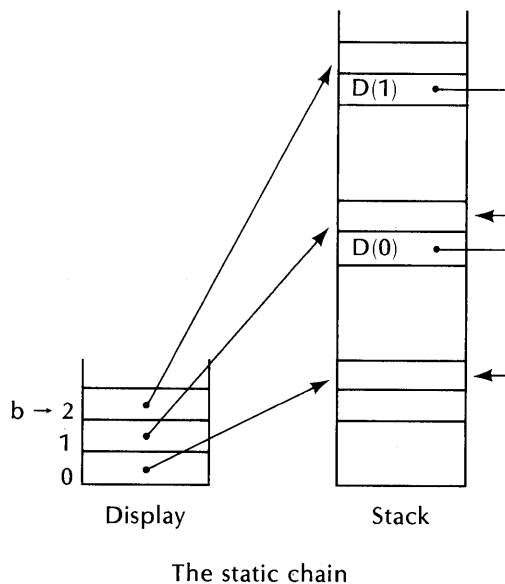
Since BE is the only operation that changes the display, we assign it (and EB) the task of building and maintaining the static chain.

BE  $n$ : Block Entry with  $n$  variables  
 $C(s+1) \leftarrow D(b)$       save static pointer  
 $b \leftarrow b+1$               increment block level  
 $D(b) \leftarrow s+2$           set display element  
 $s \leftarrow s+n+1$           reserve storage

EB: Exit block  
 $s \leftarrow D(b)-2$           reset  $s$   
 $b \leftarrow b-1$               decrement block level

Notice that since  $b$  is used, BE no longer needs its  $B$  parameter, and EB can reset  $s$  without knowing how many variables were introduced.

In the absence of procedure calls, the block index  $b$  moves up and down in an orderly fashion as blocks are entered and exited. An up-level procedure call introduces a discontinuity: the block index  $b$  jumps downward by 1 or



**Figure 9.9.** The static chain. This contains all the display information in the stack, except the topmost cell  $D(b)$ .

more. To allow RTN to reverse this jump, we must save additional information on procedure entry, namely the values of  $b$  and  $D(b)$  at the time of the call.

We therefore stack  $i$  (the return address),  $b$ , and  $D(b)$  on a call. This triple of quantities is called a *transfer point* (TP). In this context, we may think of it as a sort of generalized return address. Now we define PE and redefine CP and RTN.

```

CP  1:  Call procedure at location 1
         $s \leftarrow s + 1$ 
         $C(s) \leftarrow D(b), b, i$     save TP
         $i \leftarrow 1$               transfer

PE  B:  Procedure entry, block level B
         $b \leftarrow B$             adjust b

```

The block level  $B$  is the level of the block in which the procedure is declared. Clearly, the compiler must save this in its symbol table associated with the procedure.

```

RTN:    Return
        Go(s)    adjust display and transfer (sets i)
         $s \leftarrow s - 1$     reset s

```

Go is a procedure that accepts a transfer point location and carries out the transfer by resetting  $i$ ,  $b$ , and the display. Go is given below:

```

procedure Go(t: integer);
begin {resets display}
  var j: integer;

   $D(b), b, i := C(t);$  {unstack;
    note that  $b$  is filled before  $D(b)$ }
  j := b;
  while j > 0 do
  begin
     $D(j-1) := C(D(j)-1);$ 
    j := j - 1
  end
end

```

Usually, only some of the display elements need to be reset. If  $b'$  is the number of the smallest block which textually encloses the two points being transferred between, then  $D(0)$ , ...,  $D(b')$  are already correct and the  $b'$



assignments in the loop of Go are redundant. One could therefore reduce the number of repetitions of the loop.

The order of the instructions emitted is essential. On a procedure entry, the code should be:

```
PE B
BE n
```

On a procedure block end, the code should be:

```
EB
RTN
```

Let us now simulate the previous program in the light of our new operations. The only modifications to the program are that the instructions PE 0 and PE 1 should be inserted at the beginning of P and Q respectively, since these are the levels at which P and Q were declared. Further, the block levels can be dropped from the BE's and variable counts from the EB's.

After entering the block in which C is declared, the display/stack configuration is as shown in figure 9.10. Just before the EB in Q is executed, the configuration is as shown in figure 9.11. After execution of the EB and RTN of Q and EB of P, the configuration is as shown in figure 9.12. After executing the RTN of P, which restores the display, the configuration is as shown in figure 9.13. We see that the 7 was properly stored in X, not C, and that C's value of 3 is preserved through the calls.

Thus D(1) has been restored to its value from before the call of P.

We have now completed our version of the procedure and block operations. Although the example presented did not involve them, the following features of Algol require this complicated implementation:

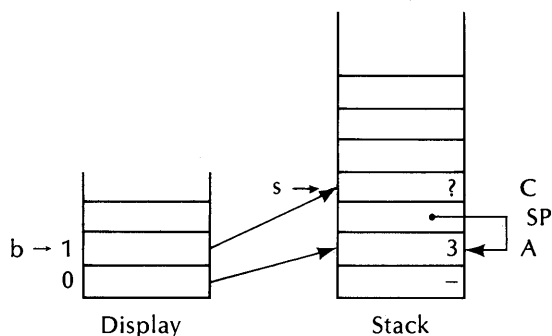


Figure 9.10. Use of the static display at run-time (see text).

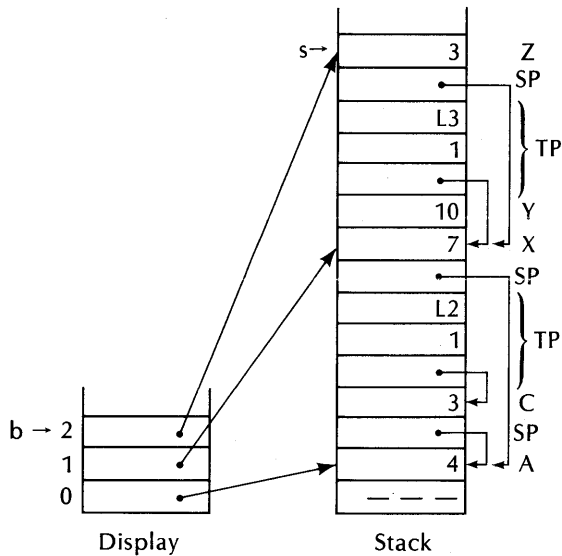


Figure 9.11. Use of the static display at run-time (see text).

1. The renaming rule, which requires that the values of free variables of a procedure are determined by the textual position of the procedure declaration
2. Recursive procedures, which require more than one activation of the same variable to coexist in general.

### 9.6.5. The Display Revisited

The display is actually redundant now, since the same information is contained in the static chain. One may therefore consider omitting the display in an implementation.

Wirth [1971a] found that three display registers were sufficient for his Pascal compiler source, and their access frequencies are as follows:

D(0): 84.2%  
 D(1): 14.7%  
 D(2): 1.1%  
 D(3): Never used

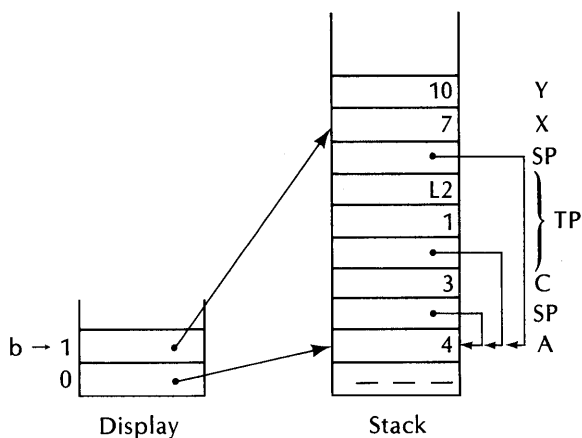


Figure 9.12. Use of the static display at run-time (see text).

This means that the Pascal compiler never exceeded a nesting level of 2 in its procedures. However, Pascal is not fully block-structured; that is, a BEGIN END pair cannot be used by itself to declare new variables, as in Algol 60, PL/I, and Algol 68. In these languages, one might expect more nesting levels to be used.

Given more nesting levels, it appears from Wirth's study and from other observations that the most frequently accessed levels are the "b" level (most

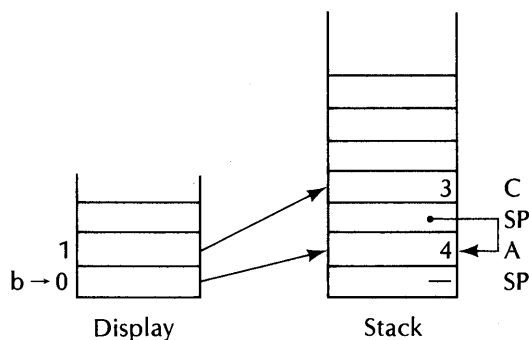


Figure 9.13. Use of the static display at run-time (see text).

local) and the “0” level (global). These levels account for most of the variable references. One can then justify a display consisting of just two registers, one for global references and another for local references. The few intermediate level references can be handled in either of two ways:

1. For every intermediate level reference, search down the static chain to the appropriate level and access the data through the indicated stack location.
2. Locate all intermediate level references in a first pass through a procedure. Then generate local addresses to each variable referenced at an intermediate level in the procedure (by following the static chain). Every data reference then is either local direct, local indirect, or global. Note that two passes are needed in this approach, since the stack allocation of local indirect labels cannot be made in the middle of expression evaluation, and a free variable can show up anywhere.

Of course, if the display is implemented in hardware, and the necessary display manipulation in firmware, as in the Burroughs 5500, its overhead cost in execution time and code is negligible.

#### 9.6.6. Labels and GOTO's

Any statement in an Algol program may be labeled. The scope of the label is the smallest textually enclosing block. A label is similar to a procedure in branching, except that a branch to a label carries no parameters and no return is expected. A branch B to some label L is legal only if the level of B is greater than or equal to the level of L, just as for a procedure.

A GOTO must reset the *i*, *b*, and *s* pointers to the context of the label. We must modify BE and EB once more to save the *s* pointer for this purpose. The saved quantity is called the *working pointer* (WP).

```

BE  n:  Block Entry with n variables
      C(s+1) ← s+n+2    save WP
      C(s+2) ← D(b)     save the static pointer
      b ← b+1
      D(b) ← s+3
      s ← s+n+2

EB:   Exit block
      s ← D(b)-3
      b ← b-1

```

Now we define an operation analogous to PE, intended to precede any branch to a label in a different block level:

LE B: Label entry in block at level B

$b \leftarrow B$

$s \leftarrow C(D(b) - 2)$

Notice that LE is a “no operation” effectively if the branch and the label are in the same block, hence could be omitted as an optimization in this case. Also, when a branch is executed, the stack configuration is expected to be exactly as it was upon entering the block containing the label, with no additional words. This means that a branch within an expression evaluation (Algol 68 permits such a thing) will cause any partial expression results on the stack to be lost. Even this loss is OK, inasmuch as branches are to complete statements, not to points within expressions.

An Algol 60 FOR loop should be implemented by stacking the step size and limit. A branch from within a FOR loop will lose this information, even if the target of the branch is within the loop. For this reason, a BE-EB pair should be executed just covering the loop code. Then a branch within the FOR loop, or even out of the loop into some covering loop, will adjust the stack properly.

The JP and JIF instructions from section 9.2.3 may still be used for the GOTO statement.

Consider the following program:

```
begin integer A;
  A:=0;
  begin integer B;
    procedure G;
      begin B:=READ;
        if B<0 then goto ENDF;
        A := A+B
      end;
    LOOP: G;
    goto LOOP
  end;
  ENDF: PRINT(A)
end
```

Its translation is:

|     |     |                 |
|-----|-----|-----------------|
| BE  | 1   | begin integer A |
| LC  | 0   | A:=0            |
| STD | 0,0 |                 |
| BE  | 1   | begin integer B |
| JP  | L2  |                 |

```

G  PE    1    procedure G
    BE    0    begin
    READ   B:=READ
    STD    1,0
    LV     1,0  if B<0 then
    LC     0
    LS
    JIF    L1
    LE     0
    JP     E    goto ENDF
L1  LV     0,0  A:=A+B
    LV     1,0
    ADD
    STD    0,0
    EB          end
    RTN
L2  CP     G    LOOP: G
    LE     1
    JP     L2   go to LOOP
    EB          end
E   LV     0,0  ENDF: PRINT(A)
    PRINT
    EB
    HALT

```

*Notes:*

1. The first JP in the program allows us to write the translation of G in the same relative position as its procedure declaration.
2. The BE-EB pair around G's translation could be omitted since no new storage is needed.
3. The EB immediately before statement E is never executed.

**Exercises**

1. The instructions CP 1 and RTN cause a branch to some other program memory location. Exactly how is this achieved?
2. Assign textual addresses to each of the variables in the following program, then translate it to AOC code:

```

begin integer A;
  integer array B[5:9],
    C[-3:2];
  integer D,E;

  procedure SAM;
  begin integer B,C;
    integer array X[0:15,3:7];

    procedure ED;
    begin integer array Q[3:9];
      integer Z;
      Z:=X[3,6];
      B:=Z+1
    end;

    procedure MAX;
    begin integer array B[0:5];
      integer Y,Z;
      Z:=B+1;
      ED;
    end;
    B:=C+3;
    if B<0 then go to ENDP;
    MAX;
  end;
  A:=B+C[B]+D-E;
  SAM;
ENDP: end;

```

3. Assume that the display D is replaced by a single register Q that effectively contains D(b) at all times. (Register b still exists.)
  - (a) Redefine each of the instructions that involve D and procedure Go, to reflect this plan.
  - (b) Give an algorithm for up-level address references, assuming that each reference requires a static chain sequential search.
4. One might suppose that procedure Go could be translated into AOC instructions, and that it could operate using the display, stack, etc. Is this so? If not, why not? Could it be written in a mixture of AOC instructions and “contents” notation, to operate on the current stack?

5. Show that LE is a “no-operation” if the branch and the label are in the same block.
6. Given that some branch does not require an LE, can this fact always be detected in a one-pass compiler? In a two-pass compiler?
7. Design instructions to implement an Algol FOR loop. The loop rules form is

```
for <var> := <expr1> [ step <expr2> ] until <expr3>
do <statement>
```

The loop rules are

- (a) The step size <expr2> may be positive or negative, but not zero. If the step is omitted, a step size of + 1 is assumed.
- (b) The <statement> cannot affect the step size or termination condition <expr3>, but can affect the <var> value.
- (c) The test of <var> value against the termination value <expr3> is made before the <statement> is executed; thus the <statement> may not be executed.
- (d) The <statement> is executed on any iteration only if the current values satisfy

$$\text{SIGN}(\langle \text{expr3} \rangle) \times | \langle \text{expr3} \rangle - \langle \text{var} \rangle | \geq 0$$

where  $\text{SIGN}(X) = \text{if } X > 0 \text{ then } +1 \text{ else } -1$ , and where  $|X| = (\text{if } X > 0 \text{ then } X \text{ else } -X)$ .

- (e) The <statement> may contain any Algol statement or block, including other FOR loops.
8. Give a reasonable set of restrictions for branches into and out of FOR loops (cf. exercise 8 above), and sketch a compiler algorithm to detect such restrictions. What stack adjustments, if any, are needed on such branches?
9. When are both a PE and a BE required on a procedure call? Give examples of programs in which (a) only a PE is needed, and (b) only a BE is needed upon entering some procedure or block.
10. Pascal permits the declaration of variables only upon a procedure entry, not upon an arbitrary block entry (BEGIN END). Are separate PE and BE instructions needed? If not, design a combined instruction.



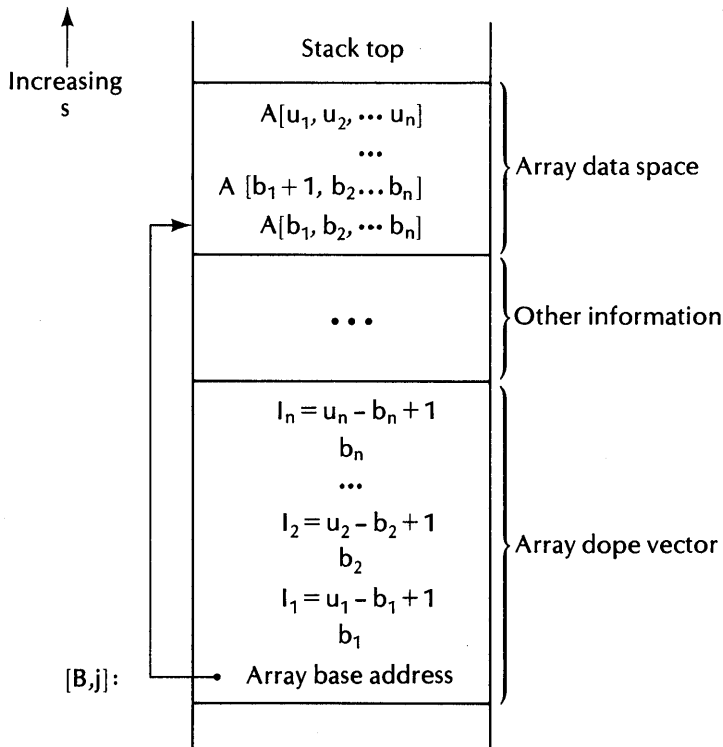
## 9.7. Arrays

Arrays can be implemented in at least two different ways. The first of these, *packed* arrays, conserves memory space at the expense of element access time. The second, *matrix pointer* arrays, requires more memory space, but provides a rapid access means.

Both systems require a *dope vector* that can be allocated by the compiler, and assigned to a transfer point  $[B, j]$ . The dope vector structure is the same for both array access means and is shown in figure 9.14. It consists of  $2n + 1$  words, where  $n$  is the number of array dimensions (the *rank*.) Although the upper and lower limits of an array may be altered dynamically, its rank is fixed at compile time.

### 9.7.1. Packed Arrays

Packed arrays are implemented through two new instructions, MAS and AVA, as follows:



**Figure 9.14.** Stack configuration upon completing allocation of a packed array through procedure MAS. The array dope vector is given, then MAS allocates the necessary space and sets the array base address in the dope vector.

MAS  $B_j, n$ : Make array space.  
 $[B_j]$  is the array base address,  
 $n$  is the array rank.

MAS allocates space for the array from the stack, based on the size and lower bound information given in the dope vector. The array base word address is also filled in by MAS.

AVA  $n$ : Array Variable Address.

AVA will calculate the address of an array element, given its subscripts and the array base word address arranged on the stack. AVA may also be used to check that the given subscripts lie within the dimensioned range of the array.

Array allocation occurs in two steps as follows:

1. While scanning the declarations in the head of a block, the compiler allocates space for the dope vector of each array. Code may be emitted to generate the lower bound ( $b_i$ ) and size ( $l_i$ ) information in the vector (see figure 9.14). The vector is arranged such that a pair ( $b_i, l_i$ ) may be computed and left behind on the stack, while parsing a declaration, from left to right, to form a dope vector. The textual address of the dope vector  $[B_j]$  and the rank  $n$  are noted and kept as array attributes in the compiler's symbol table. No space is allocated yet.

2. After all the declarations are scanned, MAS is called for each of the arrays. The allocated space will depend on how the dope vector is filled in during execution of the block-head code, hence the locations and size of the array spaces (except the location of the first) will be unknown to the compiler.

In general, the dope vector and the allocated array space will be separated by an unknown number of words. Only the dope vector transfer point is known to the compiler.

Array space and the dope vector created for the array declaration

$$A[b_1:u_1, b_2:u_2, \dots, b_n:u_n]$$

by MAS is shown in figure 9.14. For dimension  $i$ ,  $b_i$  is the least value that a subscript may take and  $u_i$  is the greatest value that subscript  $i$  may take. These may be expressions in general, containing variables previously declared and defined.

An array's data space is allocated somewhere higher in the stack than its dope vector. The array elements start with

$$A[b_1, b_2, \dots, b_n], A[b_1 + 1, b_2, \dots, b_n] \dots$$

and end with

$$A[u_1, u_2, \dots, u_n]$$

Note that the left-most index is the most rapidly varying one. We shall see that this arrangement leads to an efficient implementation.

The array base word is needed to give the array a constant transfer point despite the varying locations and sizes of the array elements. The base word is filled in during execution of its MAS instruction.

For example, consider the program

```
begin integer N;
      N:=READ; {dimension is variable}
begin integer I;
      integer array A[1:N],
                      B[3:5,N + 1:N + 15];
      .
      .
      .
```

This program translates to:

```
BE 1  begin integer N;
READ N:=READ
STD 0,0
BE 1  begin integer I;
      {"1" is the words needed for
      variable I.
      The dope vectors for A and B will be
      constructed on the fly.
      The textual addresses are:
      I: [1,0],
      dope vector A: [1,1],
      dope vector B: [1,4] }
LC 0  {dope vector A base address,
      not known yet}
LC 1  {A lower limit}
LV 0,0 {size, N}
LC 0  {B base address, fixed up later}
LC 3  {B lower limit, b1}
LC 3  {first dimension size, l1}
LV 0,0 {N}
LC 1  1
ADD   {lower limit of B, b2}
LV 0,0 {N}
LC 15
ADD   {upper limit, u2}
```

```

LV 1,7 {lower limit}
SUB
LC 1
ADD {size,  $l_2$ }
MAS 1,1,1 {allocate space for A}
MAS 1,4,3 {allocate space for B}
.
.
.

```

The MAS procedure is defined as follows. It requires the transfer point [B, j] of the dope vector and the number of dimensions N. It then allocates space for the array and sets the array base address in its dope vector.

```

procedure MAS(B, j, N: integer);
begin {make array space}
  {[B, j] = TA of dope vector,
   N = array rank}
  var DV,T,I,X: integer;
    {temporary variables}
  DV:=D(B)+j; {absolute dope
               vector address}
  C(DV):=s+1; {base address}
  X:=1;
  DV:=DV+2;

  for I:=0 until N-1 do
  begin
    X:=X*C(DV); { $1_1 \times 1_2 \times \dots \times 1_n$ }
    DV:=DV+2
  end;
  s:=s+X {the space allocation}
end

```

MAS is designed to execute at run-time. However, it cannot use the data stack of the user program without some changes. If MAS is called as a procedure, a stack marker will be established, yielding a new stack top address s. The array will be allocated using this new s, after the temporary variables of MAS have been allocated. Unfortunately, upon returning, all the allocated space will disappear, through the way the procedure return works.

If it is necessary that MAS operate on the user stack, then (1) the base address should be adjusted to reflect the known number of words added to the stack by the MAS call, and (2) just after the stack space is allocated, the MAS stack marker and its temporary variables should be moved to the top of the

allocated space, and  $D(B)$  adjusted. Then upon exit, the allocated space will in fact be left behind in the stack as wanted, and all the traces of the MAS call will have gone away.

Now consider the location of an array element:

$$A[i_1, i_2, i_3, \dots, i_n]$$

This array element is stored in the stack location

$$(\text{base location}) + i_1 - b_1 + l_1*(i_2 - b_2 + l_2*(i_3 - b_3 + \dots + l_{n-1}*(i_n - b_n) \dots))$$

The AVA instruction expects the following list on the stack:  $i_1, i_2, \dots, i_n$ , DVA, where DVA is on the stack top, and is the dope vector address. The necessary code sequence to access the array elements is therefore:

```
{evaluate  $i_1$ }
{evaluate  $i_2$ }
.
.
.
{evaluate  $i_n$ }
LA Bj {dope vector transfer address}
AVA n {n=array rank}
{array element address is
left on the stack}
```

The following procedure defines AVA. This, like MAS, needs some modifications if it is to operate on the user stack. We leave the necessary modifications to an exercise.

```
procedure AVA(N: integer);
begin {compute array element address
from stack information}
var T,U,I: integer;

T:=C(s); {dope vector address}
T:=T + 2*N - 1; {address of  $b_n$ }
s:=s - 1; {drop transfer address
from stack}
U:=C(s) - C(T); {element offset for
one-dimensional array}

I:=N - 1;

while I > 0 do
begin
s:=s - 1;
```

```

    T:=T-1;
    U:=U+C(T) + C(s) - C(T-1);
    T:=T-1;
    I:=I-1
end;
C(s):=U+C(T-1) {final element address}
end

```

In AVA, I is used as a counter to step through the dope vector, T is an address into the dope vector, and U is an element offset variable that becomes the offset from the base address of the array at the end of the while-do. Some optimizations of the while loop are suggested, since the dope vector elements are needed in just the order that they appear, moving downward in the stack.

We leave as an exercise a proof that AVA correctly computes the array element address.

#### Example

Here is a program and its translation that illustrates procedure AVA.

```

begin integer N,M;
  N:=READ;
  M:=READ;
  begin integer array A[1:N],
    B[0:M, N:20, N:M+N];
    integer I,J;
    I:=READ;
    J:=READ;
    A[I] := I+J;
    B[J,19,I+3] := A[J-1]
  end
end

```

```

BE 2  begin integer N,M
READ N:=READ;
STD 0,0
READ M:=READ;
STD 0,1
BE 0  begin ... A[], B[], I, J;
      {space will be allocated as we move along.
      The final transfer points will be:
      A d.v. 1,0
      B d.v. 1,3
      I 1,10
      J 1,11}

```

```

LC 0    base address for A
LC 1    store base,  $b_1 = 1$ 
LV 0,0  fetches  $N$ , is in fact the wanted size
LC 0    bottom of descriptor words for B
LC 0     $b_1 = 0$ 
LV 0,1  fetch M
LC 0
SUB
LC 1
ADD      $u_1 - b_1 + 1 = M + 1$ 
LV 0,0   $b_2 = N$ 
LC 20    $u_2$ 
LV 0,0
SUB
LC 1
ADD      $u_2 - b_2 + 1 = 21 - N$ 
LV 0,0   $b_3 = N$ 
LV 0,1
LV 0,0
ADD
LV 0,0
SUB
LC 1
ADD      $u_3 - b_3 + 1 = M + 1$ 
INCS2   space for I, J
MAS 1,0,1 create space for A
MAS 1,3,3 create space for B
READ    I:=READ;
STD 1,10
READ
STD 1,11 J:=READ
LV 1,10 I
LV 1,0  array base word for A
AVA     get address of A[I]
LV 1,10 I
LV 1,11 J
ADD
ST      A[I] := I+J
LV 1,11 J
LC 19
LV 1,10 I
LC 3
ADD
LV 1,3  array base word for B

```

```

AVA      get address of B[...]
LV  1,11
LC  1
SUB
LV  1,0  array base word for A
AVA      get address of A[J-1]
CONT     get contents of A[J-1]
ST       B[J,19,I+3] := A[J-1]
EB
EB
HALT

```

### 9.7.2. Array Access Through Matrix Pointers

Given an array of rank  $n$ , we may organize the array access by constructing *pointer vectors* in the stack, as well as allocating space for the array elements. The determination of an array element address is much faster with pointer vectors than through the index calculations of AVA above, if the rank is two or more. If the rank is one, the two methods are essentially the same.

Let us begin with access. For rank  $n$ , and an array element of the form

$$A[i_1, i_2, i_3, \dots, i_n]$$

we construct a dope vector exactly as before at the transfer point  $[B,j]$ . (However, the base address and allocation is handled by procedure MMAS, given below.) Then the following code sequence is emitted:

```

LV  B,j  {this will be the
          base address, less  $b_1$ }
{evaluate  $i_1$ }
ADD     {end of one index}

CONT     {for a second index}
{evaluate  $i_2$ }
ADD     {end of second index}

.
.
.
CONT     {for the  $n$ th index}
{evaluate  $i_n$ }
ADD     {element address will be
          on the stack}

```

The simplicity of this scheme speaks for itself. Instead of building a list of



indices, then calling AVA, the simple instructions CONT (indirection) and ADD are used to determine the address.

A conceptual diagram of this access scheme is given in figure 9.15. Only one dope vector element, the base address, is needed. The base address of the dope vector points to a fictitious memory location,  $b_1$  words below a vector of  $l_1$  pointers. Note that the first index  $i_1$ , must be at least  $b_1$ , and is added to the base address during access. After the first ADD, we therefore have the address of one of the  $l_1$  pointers. CONT fetches its contents, and this is the address of another fictitious memory location,  $b_2$  words below a vector of  $l_2$  words. At the end of the access sequence, we have the address of an element.

A specific stack configuration for the declaration

var X: array [2:3, 3:2] of integer

with the dope vector X address 11 and the top of stack address 20 upon calling procedure MMAS, is shown in figure 9.16. The dope vector is in locations 11/15, the first (and only) matrix pointer vector is in locations 21/23, and the

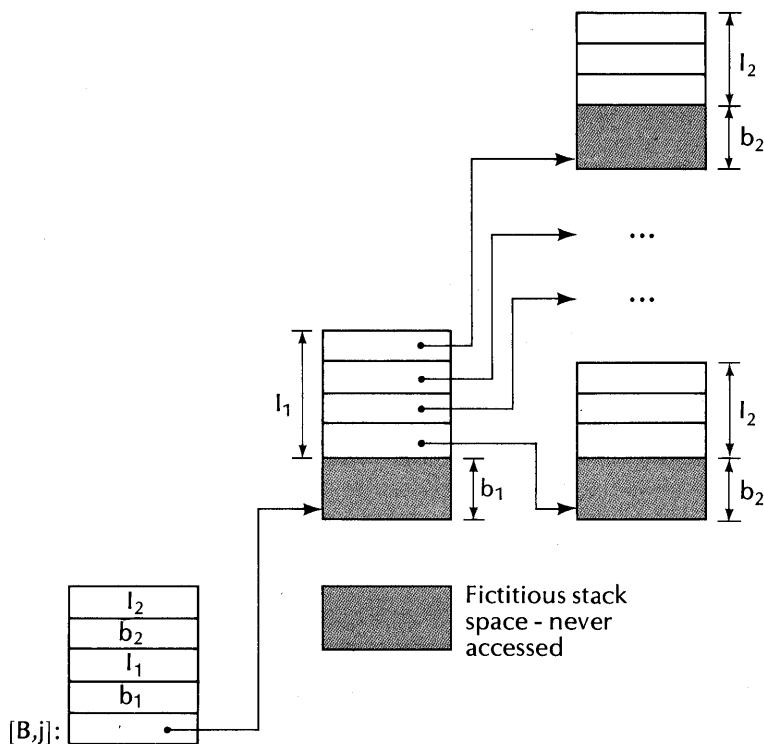
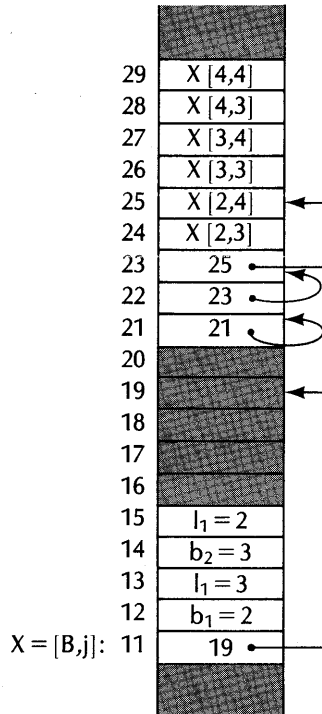


Figure 9.15. Conceptual stack configuration for matrix-pointer array access.



**Figure 9.16.** Actual stack configuration for matrix-pointer array access, just after MMAS has allocated the necessary space and set up the pointers.

data elements are in locations 26/29. The configuration is exactly as MMAS, given next, would generate.

Now let us define procedure MMAS. This sets the base address for an array, allocates space for it, and sets the pointer vectors appropriately.

```

procedure MMAS(B, j, N: integer);
begin {make matrix array space.
      [B, j]: dope vector
          transfer address,
          N: array rank}
var DV: integer;

procedure SETP(N,A,D: integer);
begin {set pointers}
var I: integer;

if N>0 then
begin {return if N is zero}
for I:=0 until C(D-1)-1 do

```

```

begin
    C(A+I):=s+1-C(D);
        {set one pointer}
    s:=s+C(D+1); {allocate a
        vector}
    SETP(N-1, s-C(D+1)+1, D+2)
        {then fill the vector}
end
end
end; {of procedure SETP}

DV:=D(B)+j; {absolute dope
    vector address}
C(DV):=s+1-C(DV+1);
    {base address, less offset b1}
DV:=DV+1;
s:=s+C(DV+1); {space for a vector}
SETP(N-1,s-C(DV+1)+1, DV+2)
    {fill the space}
end {of procedure MMAS}

```

Procedure SETP is the key to understanding MMAS. When SETP is called, it expects that a contiguous array of words, starting at address A, has been allocated on the stack, but not filled. It also expects D to be the address of the next dimension pair in the dope vector. The purpose of the FOR loop is to fill this vector with the addresses of spaces that SETP is allocating. Note that when N reaches 0, the space at address A is simply allocated and not filled; this becomes data space.

If the FOR loop instructions are executed once or more times, each iteration places one address in the vector. The address is the next available stack location  $s+1$ . After the address is placed, space is allocated for the vector (or data) and SETP is called to fill it, based on the next dimension (at location  $D+2$ ). The recursive calls of SETP effectively set up an entire matrix of pointers and array spaces before returning to set up the next pointer in its list. At the end of a set of recursive calls is an allocation of uninitialized array space. The maximum depth of recursive calls is  $r$ , the number of dimensions of the array. However, the total number of calls is  $n_1 \times n_2 \times \dots \times n_{r-1}$ , or 1 if  $r=1$ . This of course can be a large number of calls. However, MMAS is only called upon a declaration. Access of an array element is fast inasmuch as no multiplications are needed.

MMAS will result in sets of data space and pointer vectors intermixed in the stack in some fashion. It is obviously essential that array indices lies within their declared bounds, otherwise data values will be treated as pointers, or pointers will be overwritten with data. Clearly, array access can

be embellished with array bounds tests, although at some increase in access code and execution time.

Procedures MMAS and SETP expect their parameters to be passed by value. This means that a local copy of the parameter value is built on a stack. The copy may therefore be used and modified without affecting the original value. The FOR loop in procedure SETP must carry its limit value on a stack, as well. Of course, the local variable I in SETP is stacked.

MMAS cannot be executed on the user stack without some modifications to prevent loss of the allocated space and pointers upon return. We leave details to an exercise.

### 9.7.3. Dynamic Arrays and Redimensioning

An Algol 60 variable may be designated OWN or be considered local by default. A local array is allocated by one of the previous schemes upon entering a new block and is deallocated upon exiting from the block. Deallocation is automatic—as the stack top pointer *s* is simply reset to its value upon entering the block. There are also no language features that permit the dimensions of a local array to be changed within the block (at least not in Algol or Pascal. However, PL/I permits arbitrary redimensioning of type VARYING arrays at execution time). The dimensions may be arbitrary expressions containing variables defined in some covering scope. Such a dynamic array can be allocated from the stack by MAS or MMAS, operating at run-time.

Now consider an OWN array. The values of an OWN array must be preserved between an exit from its block through a subsequent re-entry into the block. This alone means that an OWN array cannot be allocated on the stack top; another data space, the heap, is needed for OWN variables and arrays.

If the OWN array has dimensions that are fixed at compile time, the compiler can simply allocate a group of words in the heap for the array. This obviously applies to simple OWN variables as well. Unfortunately, the Algol 60 rules permit redimensioning OWN arrays. The redimensioning cannot change the rank, but it can change the upper and lower limits. The change, of course, occurs if any of the dimensions is a variable expression. The Algol 60 rules furthermore specify that upon such a redimensioning, the value of every data element whose index set is valid under both the old and new dimensions must be preserved. That is, if the array element

$$A[15,7,20]$$

carries valid indices under both the old and the new dimensions, then its value must be preserved.

The new dimensions may be such that the overall array size is increased;

the new array will then not fit into the old allocated heap space. The old space must be released to a pool of available heap space, and new space must be allocated. All the compatible data values must also be copied into the new space.

If the new data space size is less than the old, then the old data space can be re-used, with some space possibly left over, available for other uses. Data values must in general be moved to new locations. The easiest way to move the values is to first copy them to a temporary heap or stack space, then copy them back into their correct positions. However, Ingberman [1961] solved the problem of moving the data values correctly in place, without requiring more than a few temporary memory cells.

The dynamic allocation of new heap space and the discarding of existing space calls for a memory management system for the heap in any practical implementation. We have regarded the heap and stack space as infinite, but of course in any implementation these spaces are limited by available memory and by fixed address word sizes. A limitation on stack space is not necessarily fatal to a program, since stack space tends to be reclaimed; no corpses of memory are left around. Heap space on the other hand will eventually consist of large dead areas separated by live areas. When a memory limit on heap space is reached, something should be done to make use of the dead areas, in an attempt to keep the program going, at least until all the available heap memory is filled with live data.

There is general agreement that the most efficient memory management system for this application is one in which new heap space is allocated from *h* downward, as needed. When a heap memory limit is reached, the program is interrupted, and the heap space is collapsed by collecting together all the live areas. This process is called *garbage collection*. The garbage collector must be able to determine the locations and sizes of each of the areas and whether an area is dead or live. It is sufficient to provide two pointers PREV/HEAP and BACK/PNTR and a flag LIVE in each data area, assuming that new areas are always allocated in the next heap area and are linked (with PREV/HEAP) in the order last-allocated-first. Then the first pointer value is also one word more than the end of the data area containing the pointer. (The purpose of the pointer BACK/PNTR will be clear in a moment.)

Given such a chain, the garbage collector would first build a list of live areas and their lengths on the stack by following the PREV/HEAP chain. It would then move the live areas to the heap bottom, fixing their links appropriately, in link order. Dead areas are thereby removed in this process.

The garbage collector must also update all references to the live areas. The update can be managed by requiring that exactly one pointer to a live heap area exist, on the stack. Let its location be *P*. We then need a pointer BACK/PNTR in each heap data area to *P*, so that the garbage collector can locate and adjust the pointer in *P*.

We see that each heap data area must therefore contain the following data structure—the remainder of the area is available for the data.

```
var PREV'HEAP, BACK'PNTR: integer;
    LIVE: Boolean;
```

Garbage collection can be implemented as a parallel process through a system of semaphores. See Gries [1977], Steele [1975], and Dijkstra [1976a] for details. Such an implementation would be very efficient if an independent processor could be dedicated to garbage collection. Those heap data areas not currently in use can be moved into dead areas while other processing continues concurrently.

#### 9.7.4. Pascal Data Structures

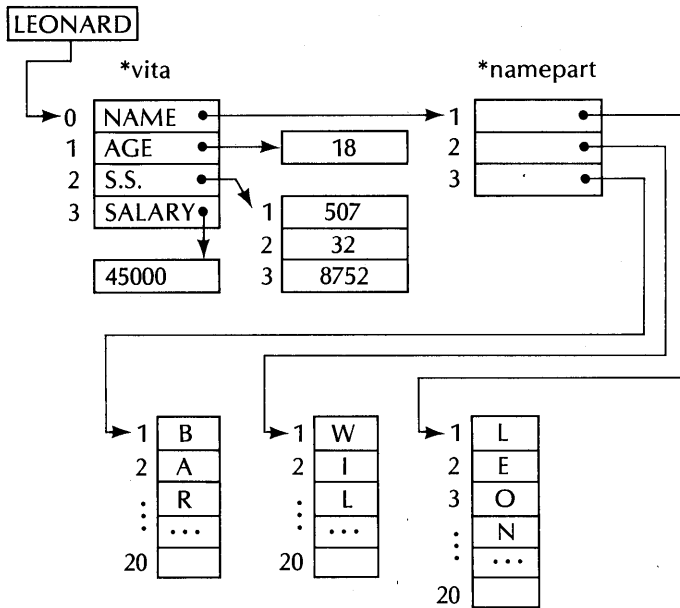
We next consider the problem of allocation of Pascal data structures through AOC operations. This problem is a kind of generalization of the array access systems described in the previous section and in fact admits of two access solutions, one with direct index calculations and another with pointers.

These two kinds of structure organization are illustrated in Figs. 9.17 and 9.18. They correspond to the Pascal declarations

```
type NAMEPART=array [1..20] of char;
type NAME=array [1..3] of NAMEPART;
type VITA=record NAME: NAME;
                AGE: integer;
                SS: array [1..3]
                  of integer;
                SALARY: integer;
            end;
var LEONARD: VITA;
```

Figure 9.17 shows a matrix pointer implementation of LEONARD. At run-time, LEONARD is a pointer to a VITA list. VITA contains pointers to NAME, AGE, SS, and SALARY. Access to some primitive object at the end of a chain requires indirection, indexing, indirection, indexing, . . . , just as for array matrix pointers.

Figure 9.18 shows a packed implementation of LEONARD. Here the required data areas are packed into a contiguous group of data memory words, referenced by a textual address associated with LEONARD. The



**Figure 9.17.** Run-time support structure for a Pascal VITA structure, using pointer matrices.

necessary offset of some data element is in general an expression whose operands are some combination of offsets within the structure and indices. Since the indices are run-time variables in general, the offset must be computed at run-time. Notice that a dope vector is unnecessary; each of the data items in figure 9.18 has a position known to the compiler, because the dynamic dimensioning of such structures is not supported in Pascal.

Let us begin with some observations that apply to Pascal data structures (and Pascal only!):

1. Every declared type has a fixed size.
2. Every type appearing in a structure bears some offset from its ancestor types, known at compile time. However, the offset depends on the ancestor. Recall that a given type may appear in more than one declaration and will therefore carry an offset that depends on the declaration details.
3. A type is allocated space and assigned an address only upon an appearance in a “var” or “constant” declaration. Until such an appearance, a Pascal type is an abstract description of a certain configuration of data.

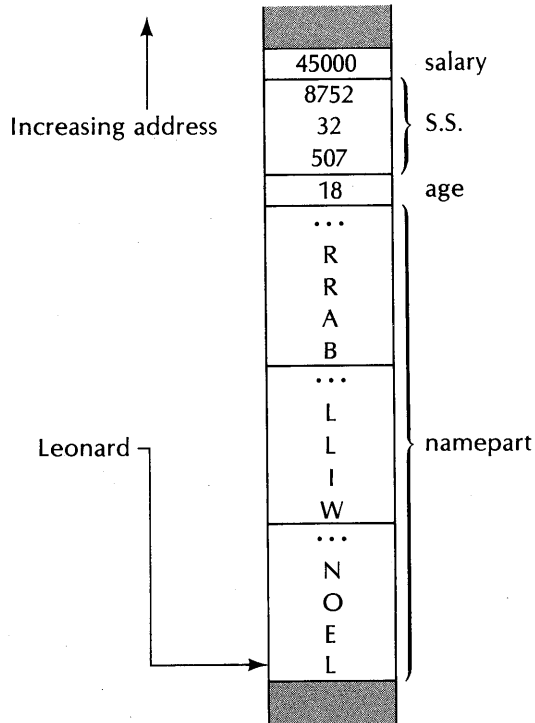
We conclude from these observations that the compiler can assign a textual address *T* and allocate space to each declared variable name, e.g., *ED* in

var *ED*: <some type name>;

However, the address of each component datum in ED must be worked out on each reference. Only the variable ED can be assigned an address. The components of ED in the symbol table type structure will in general be shared by several different variables with different locations.

The mechanism of developing an address for a component of some variable is given by the following algorithm.

1. The *root name* is the name declared in a VAR or CONST declaration. It is assigned a transfer point  $[B, j]$  by the compiler. The next available transfer point is  $[B, j + j']$ , where  $j'$  is the number of words needed for the structure associated with the root name.
2. The size  $j'$  of a structure is determined by the compiler through a bottom-up tree walk through its structure lists, as follows. Each element in the structure carries a pair  $\{S, D\}$ ;  $S$  is the size of the object and  $D$  is its offset relative to a related object.
  - (a) For a primitive data object,  $\{S, D\} = \{\text{size of the object}, 0\}$ .
  - (b) For an array object, the child will carry the pair  $\{S, D\}$ . If the upper



**Figure 9.18.** Run-time support structure for a VITA structure, using packed configuration.



and lower bounds of the array are  $U$  and  $L$ , then the array object will carry the pair  $\{S*(U-L+1), D-L\}$ .

- (c) For a RECORD node, with selectors  $s_1, s_2, \dots, s_n$ , and descendant types  $T_1, T_2, \dots, T_n$  (e.g. figure 8.6), let selector  $s_i$  carry the pair  $\{S_i, D_i\}$ . Then  $D_1=0, D_2=D_1+S_1, D_3=D_2+S_2$ , etc. In general, each of the  $D_i$  except the first will change from their original value of 0. Then  $S$  for the record node will be  $D_n+S_n$ , and  $D$  for the node will be 0.
- (d) For a variant node (CASE), each of the case fields will carry some displacement  $S'$ , where  $S'$  is the size of the CASE data object. The displacement  $D$  of the variant node will be 0, and its size will be  $S'$  plus the maximum of the sizes of the field sizes. Thus the variant type structure is supported by an allocated memory space sufficiently large for the largest possible case.

For example, here are the TA's assigned to the components of LEONARD (cf. figure 8.8 and figure 9.18):

| Component           | Offset $j$ in TA $[B,j]$ |
|---------------------|--------------------------|
| LEONARD.NAMEPART(0) | 0 bytes                  |
| LEONARD.AGE         | 60                       |
| LEONARD.SS(0)       | 62 {an integer=2 bytes}  |
| LEONARD.SALARY      | 68                       |

The size of the LEONARD structure is 70 bytes.

### 9.7.5. Pascal Data Object Access

A structured data object is accessed by refining the base address  $[B,j]$  of its root node. Now each of the primitive data objects in the structure will carry some offset  $j'$  from the root address. However, because of array indexing, the offset will in general be a run-time value. We propose a scheme for determining  $j'$  for some data object described by a compound Pascal name. The scheme will emit code that locates the item, and if the code evaluates to a constant, then presumably the compiler will be able to identify this fact.

The emitted code might first be expressed as an AST of course, which permits various optimizations and reductions.

Given the base address  $[B,j]$  of a data object, we then consider the path downward through the type structure associated with the object (cf. section 8.3.3.3). Offset  $j$  is then altered as follows in passing through various nodes:

1. In passing through an array node with the index  $X$  (possibly an expression), lower dimension  $L$ , and pair  $\{S,D\}$ ,

$$j := j + S*(X-L) + D$$

Effectively, the array blocks of size  $S$  each begin at displacement  $D$  relative to their father node.

2. In passing through a record selector node associated with the pair  $\{S, D\}$ ,

$$j := j + D$$

3. In passing through a variant node, with pair  $\{S, D\}$ ,

$$j := j + D$$

When the data object at the end of the search path is reached, its transfer point is  $[B, j]$ .

For example, LEONARD.SS(2) in figure 9.18, with a packed structure on byte boundaries and with 2-byte integers, will be accessed as follows:

- Let  $A' =$  byte address of the LEONARD structure.
- Then  $A' + 62 =$  byte address of the SS part of the structure (three 20-byte nameparts and one 2-byte age precedes this address)
- Then  $A' + 62 + 2 \cdot (2 - 1) = A' + 64 =$  byte address of LEONARD.SS(2).

### Dynamic Pascal Objects

A pointer in Pascal is associated with some data structure. If pointer  $P$  is associated with type  $T$ , then  $P$  can only be set to point to a structure of type  $T$ . New memory space can be allocated to a pointer  $P$ , through the function *new*( $P$ ) This procedure allocates sufficient space (in the heap) for the type associated with  $P$  and also sets  $P$  to point to it. The procedure *new* is useful for building repeated data structures such as trees and lists.

We have already discussed the problem of dynamic allocation of memory space in the heap. We need only discuss the access of a data object through a pointer which is achieved essentially as follows:

```

LV   P    {contents of pointer, an address}
      {evaluate offset of data object}
ADD
      {address of data object is now on stack}

```

### Exercises

1. Translate the following program to AOC code, using MAS. Give another translation using MMAS:

```

begin integer X;
  X:=READ;
  begin integer N;
    integer array A[0:X],
                  B[1:X,X+1:2*X];
    N:=READ;
    A[N+1]:=B[READ, READ+X]+25
  end
end

```

2. What modifications to AVA would provide run-time checking of array bounds? Consider two kinds of check: an overall check that the final addressed element is within the array somewhere, and a more particular check that each index is within its dimension.
3. What modifications to the matrix access system are needed for run-time checking of array bounds?
4. What modifications of AVA are needed for it to be called as any other AOC procedure, using the data stack, display, etc., of the calling procedure?
5. What modifications to MAS and MMAS are needed for these to operate on the user stack?
6. In figure 9.16, show that each of the six X values are correctly accessed through the access mechanism given.
7. Construct the stack configuration for the declaration

var X: array [2:2, 3:4, -3:-2] of integer

using MAS and MMAS. Let the dope vector reside at address 75 and stack top address  $s = 123$  upon calling MAS or MMAS.

8. Compare the performances of AVA and matrix pointer access for an array of rank  $n$ . Assume that each memory fetch and store, that each unary or binary stack operation (other than multiply) requires 1 time unit, and that a multiply requires 5 time units.
9. Which requires more operations—MAS or MMAS? Does your answer depend on the rank or on the array dimensions?
10. Write a heap garbage collector in Algol, Pascal, or AOC code. Could this operate as a procedure on the user stack? When should it be invoked? Assume that the system imposes an absolute upper bound on

[h]. Are any new AOC instructions needed (i.e., in addition to those already introduced)?

11. Show the stack configuration resulting from the Pascal variable

```
var ADDR: ADDRESS
```

(cf. section 8.3.3.3), using packed accessing. Then give optimized AOC code to compute the addresses of each of its constituents.

## 9.8. Typed Procedures and Procedure Parameters

Programs are much clearer, more precise in meaning, and easier to write and maintain if the inputs and outputs of procedures are explicitly declared. An Algol procedure declaration has the general form

```
<type> PROCEDURE <name>
  ( <fp1> , <fp2> , . . . , <fpn> );
  <VALUE fp declarations> ;
  <specification fp declarations> ;
<procedure-body>
```

The <fp<sub>i</sub>>'s are simple identifiers, called *formal parameters*, or FP's. Their attributes are declared in the <VALUE fp declarations> and in the <specification fp declarations>. The former list specifies those that are passed by VALUE; all others are passed by name. The latter list specifies the type (REAL, INTEGER, etc.) and whether an ARRAY or not.

The <procedure-body> is a block, consisting in general of a BEGIN-END pair enclosing local declarations, more procedure declarations, and a list of executable statements. Algol 60 also permits a single executable statement, without a BEGIN-END pair, as a <procedure-body>.

The procedure <type> must appear if the procedure is to be used as a function; this is the type of the returned value, REAL, INTEGER, etc. The value of a typed procedure is assigned-to within the <procedure-body> by a statement of the form

```
<procedure name> := <expression>
```

The parameters  $e_1$ ,  $e_2$ , etc. that appear in a call

```
<procedure name> (  $e_1$ ,  $e_2$ , . . . )
```

are called *actual parameters* or AP's. A procedure call is handled roughly as follows in the AOC machine:

1. If the procedure is typed, a word is allocated on the stack for its return value. Upon a procedure exit, this word will be left in place, exactly as needed for whatever expression evaluation was taking place at the moment of the call.
2. For each of the actual parameters, the compiler considers the relation of the actual parameter to the corresponding formal parameter declarations. Algol 60 permits passing parameters by *value* or by *name*. Other parameter passing mechanisms have been proposed—we will consider passing by *reference* later. The compiler must emit code that will cause a value, an address, or something to be written on the stack for each actual parameter.
3. Finally, the procedure is called through the CP instruction. The next execution step is then the first line of the <procedure-body> code. Within the procedure, the material previously written on the stack as actual parameters is now viewed as associated with the formal parameters. They are accessed through textual addresses of the form [B,j], where j is negative. Upon an exit (RTN) from the procedure, this material must be removed, leaving at most the procedure return value on the stack. Adjustments to the display are also made, and the next executed statement is the statement following the procedure call in the calling instruction sequence.

Certain abstract rules govern the semantics of procedures and procedure calls. These are the *procedure copy rule* (PCR), rules for *call by value* (CBV), for *call by name* (CBN), and a *typed procedure exit rule* (TPER). These rules specify the semantic effect of calls.

### 9.8.1. The Procedure Copy Rule PCR and Call by Name CBN

The execution of a (typed) procedure call

$$P(e_1, e_2, \dots, e_n)$$

is effectively (but not in practice) carried out by:

1. Replacing the call by a copy of the body of the procedure declaration, using TRR to avoid variable conflicts.
2. Replacing all occurrences (in the copy) of the formal parameters  $x_1, x_2, \dots, x_n$  by the corresponding actual parameters enclosed in parentheses, i.e.,  $(e_1), (e_2), \dots$ . Again, TRR is invoked to avoid variable name conflicts.

These steps are taken before the actual parameters are evaluated. For example, a simulation of the program

```

begin integer A, B;
  procedure Q(X,Y); integer X,Y;
  begin X:=Y*Y
  end;
  B:=3;
  Q(A,B+6)
end

```

would lead to the program

```

begin integer A=?, B=3;
  procedure Q(X,Y); integer X,Y;
  begin X:=Y*Y
  end;
  B:=3;
  begin A:=(B+6)*(B+6) end
end

```

Thus the computation of  $B+6$  would be delayed until the formal parameter appears; it would also be done for each appearance, twice in this example. The value of  $A$  is also changed by the procedure call.

It should be obvious that a compiler cannot implement CBN by substituting strings in the source—the specific substitution depends on which of several possible procedure calls are being executed. Furthermore, the source has disappeared when the program is executed. We shall describe an equivalent run-time system for implementing CBN in section 9.8.7.

### 9.8.2. Call by Value (CBV)

Call by value, or CBV, is defined by the following rule:

**Value Parameter Evaluation Rule (VPER).** A procedure declaration such as

```

procedure P(A,B,C); value A,C;
  integer A,B,C;
  begin . . . end

```

is equivalent to

```

procedure P(A',B',C');
  integer A', B', C';
  begin integer A,C;
    A:=A';
    C:=C';
    begin . . . end
  end

```

where  $A'$  and  $C'$  are new variables not appearing in “begin . . . end”.

The VPER is just a formal way of stating that CBV parameters are evaluated at the time of the call and stored in new locations local to the procedures. Only the local copies are affected by the procedure instructions. An assignment to a CBV FP will change the local copy but not the AP outside the assignment.

The following example illustrates VPER and TPER:

```
begin integer A;
  integer procedure S(X); value X;
    integer X;
    begin S:=X+1 end;
    A:=3;
    PRINT(S(A*2))
end
```

and is equivalent, by VPER and TPER, to

```
begin integer A;
  integer procedure S(X'); integer X';
  begin integer X;
    X:=X';
    begin S:=X+1 end
  end;
  A:=3;
  PRINT(S(A*2))
end
```

Through simulation by copying, we arrive at the next program state:

```
begin integer A=3;
  integer procedure S(X'); integer X';
  begin integer X;
    X:=X';
    begin S:=X+1 end
  end;
  A:=3;
  PRINT(begin integer X=6;
    X:=(A*2);
    begin S:=X+1 end
  end)
end
```

Of course, the “begin . . . end” material within PRINT is syntactic nonsense

in Algol 60, but the spirit of the operations is clear. The application of TPER then yields

```

begin integer A=3;
  integer procedure S(X'); integer X';
  begin integer X;
    X:=X';
    begin S:=X+1 end
  end;
  A:=3;
  PRINT(7)
end

```

### 9.8.3. More About Free Variables

Recall that we found it important to apply TRR before TPCR in implementing procedure calls, in order to resolve the free variable problem. We can illustrate another facet of the free variable problem with the following example:

```

begin procedure G(X); integer X;
  begin integer I;
    I:=X-1;
    if X=0 then PRINT(X)
      else G(I)
    end;
  G(2)
end

```

Notice first that no applications of TRR are necessary, since the variables are all distinct. In simulating this program with TPCR, we arrive at the state:

```

begin procedure G(X); integer X;
  begin integer I;
    I:=X-1;
    if X=0 then PRINT(X)
      else G(I)
    end;
  begin integer I=1;
    I:=2-1;
    if I=0 then PRINT(2)
      else G(I)
    →

```



If we apply TPCR again, but neglect TRR, we arrive at

```

begin procedure G(X); integer X;
  begin integer I;
    I:=X-1;
    if X=0 then PRINT(X)
      else G(I)
  end;
  begin integer I=1;
    I:=2-1;
    if I=0 then PRINT(2)
      else begin integer I=?;
        I:=I-1;
        if I=0 . . .
→

```

Whereas, using TRR to make the I's distinct, we get

```

begin procedure G(X); integer X;
  begin integer I;
    I:=X-1;
    if X=0 then PRINT(X)
      else G(I)
  end;
  begin integer I=1;
    I:=2-1;
    if I=0 then PRINT(2)
      else begin integer I1=?;
        I1:=I-1;
        if I1=0 then PRINT(I)
          else G(I1)
→

```

The outcome clearly depends on applying TRR before TPCR. In the former case, the outcome is undefined since I is undefined, whereas in the latter case, "1" is printed.

### 9.8.4. Fortran Parameter-Passing Rules

The Fortran parameter-passing rules are neither CBV nor CBN, rather something in-between, called *call by reference*, CBR. A reference (memory address) is always passed. The reference will be a pointer to a variable, to an indexed variable (pointer to the variable offset by the index value), to a constant, or to some temporary assigned to the value of an actual parameter expression. An actual parameter expression is first evaluated, then assigned to a temporary memory location; the passed reference is to the temporary.

For the purposes of parameter passing, a variable or indexed variable is not considered an expression.

We see that the rule for actual parameter expressions is CBV, while the rule for other parameters is CBN. The constant rule is neither and depends on the implementation in general. In some implementations, a temporary copy of the constant is made and an address of that copy is passed. In others, the address of the constant itself is passed. In the latter implementations, the following program illustrates a run-time bug that yields a surprising and dangerously unpredictable result:

```
SUBROUTINE MIKE(X)
  X=6
END

MIKE(7)
PRINT(7)
```

Here, if only one “constant” 7 exists in memory, and its address is passed to MIKE, the constant 7 is turned into a 6, and “6” is printed. However, there might be several copies of 7, or a temporary might be assigned to 7, in which case “7” would be printed. A Fortran compiler can protect a programmer to some extent by making a temporary copy of the constant, then passing the address of the copy. At least then the constant as seen outside the procedure will not be changed by the procedure. However, something is wrong with the programmer’s concept of the procedure’s application, and the compiler cannot bring this to his attention.

### 9.8.5. Implementation of CBV and Typed Procedures

CBV is much easier to implement than CBN, so we shall consider it first. Although CBV is defined in terms of CBN, we shall develop a direct implementation that does not depend on CBN.

No new operations are required. We only need to modify RTN. The general procedure call rule is simple: when an actual parameter is to be stacked, evaluate it and leave its value behind on the stack. Within the called procedure, the value is accessed by a negative offset (known to the compiler) relative to the base of the stack marker (D(B)). Since the compiler will not otherwise know the location of the stack marker base, we require that a BE always be executed upon a procedure entry.

The stack marker then always contains five words as follows:

|                 |           |             |
|-----------------|-----------|-------------|
| static pointer  | D(b)      | (from CP)   |
| display top     | b         | (from CP)   |
| return address  | i         | (from CP)   |
| working pointer | S + n + 2 | (from BE n) |
| static pointer  | D(b)      | (from BE n) |

(Note: PE changes  $b$  and comes between CP and BE. Hence the static pointers are different in general. Thus  $sm = 5$  in the following discussion.)

The return command RTN must remove the formal parameter material as well as the stack marker, so we redefine the command as follows:

```

RTN  m:  Return from procedure
        m=formal parameter words
      Go(s)  {reset the display}
      s ← s - m - sm {reset s; sm=marker size}

```

The actual parameters are evaluated from left to right, so that in the call

$$P(e_1, e_2, e_3)$$

the value of  $e_1$  is deepest in the stack. This call yields the display and stack configuration shown in figure 9.19.

The textual addresses (TA's) required in the procedure body, at block level  $N$ , for the FP's corresponding to  $e_1$ ,  $e_2$ , and  $e_3$  are

```

for  $e_1$ :  $[N, -3 - sm]$ 
for  $e_2$ :  $[N, -2 - sm]$ 
for  $e_3$ :  $[N, -1 - sm]$ 

```

In general, the TA for  $x_i$  in the procedure with the declaration

```
procedure  $P(x_1, x_2, \dots, x_i, \dots, x_m); \dots$ 
```

is  $[N, i - m - sm]$ . Note that  $(i - m - sm)$  can be assigned by the compiler as soon as the “)” of the formal parameter names is scanned. In practice, different kinds of parameters may require different numbers of bytes each, so

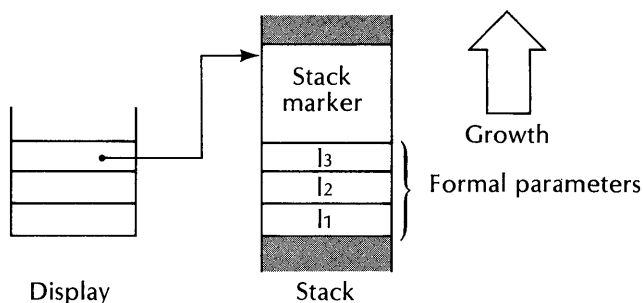


Figure 9.19. Stack configuration upon procedure call  $P(e_1, e_2, e_3)$ .

that the formal parameter declaration must be scanned before addresses can be assigned to the formal parameters.

### 9.8.6. Typed Procedure Return Value

A typed procedure can be called as a stand-alone statement, or as part of an expression, e.g.

```
P(5,Y);    {statement form}
A:=15+P(5,Y); {expression form}
```

In either case, the compiler reserves one word on the stack, perhaps via LC 0, before loading any parameters. Inside the procedure, any assignment to its name is treated as an assignment to a variable with the TA  $[N, -m - sm - 1]$ . Upon an exit, the return value will be the only remaining object on the stack. Furthermore, nothing in the stack deeper than the return value will be affected by the procedure, except through specific up-level address references in the procedure.

#### CBV example

Consider the following program, introduced previously:

```
begin integer A;
  integer procedure S(X); value X; integer X;
  begin S:=X+1 end;
  A:=3;
  PRINT(S(A*2))
end
```

This program will translate to

|   |     |      |                                    |
|---|-----|------|------------------------------------|
|   | BE  | 1    | begin integer A;                   |
|   | JP  | L    | {to get around the procedure code} |
| S | PE  | 0    | integer procedure S(X); . . .      |
|   | BE  | 0    | begin                              |
|   | LV  | 1,-6 | S:=X+1 {sm=5}                      |
|   | LC  | 1    |                                    |
|   | ADD |      |                                    |
|   | STD | 1,-7 | {return value}                     |
|   | EB  |      | end                                |
|   | RTN | 1    |                                    |
| L | LC  | 3    | A:=3                               |
|   | STD | 0,0  |                                    |

|       |     |                           |
|-------|-----|---------------------------|
| LC    | 0   | {reserve return location} |
| LV    | 0,0 | A*2                       |
| LC    | 2   |                           |
| MULT  |     |                           |
| CP    | S   | S(A*2)                    |
| PRINT |     | PRINT(S(A*2))             |
| EB    |     |                           |
| HALT  |     |                           |

### 9.8.7. Implementation of CBN Procedure Parameters

A CBN parameter will be implemented by creating a procedure that can evaluate the actual parameter, then passing a reference to the procedure. Such a procedure is called a *thunk*, a name devised by P. Z. Ingerman.

For example,

```
begin integer procedure SUM(F,N);
  value N; integer N;
  integer procedure F;
  begin integer I,S;
    I:=1; S:=0;
  LOOP:
    if I<N then
      begin S:=S+F(I);
        I:=I+1;
        go to LOOP
      end;
    SUM:=S
  end;
  integer procedure SQ(X); value X;
  integer X;
  begin SQ:=X*X end;
  PRINT(SUM(SQ,READ))
end
```

will print the sum of the squares of the integers returned by READ.

```
READ
Σ i2
i=1
```

To implement CBN procedure parameters we need two new operations:

```

LL 1: Load label 1 as a TP
       $s \leftarrow s+1$ 
       $C(s) \leftarrow D(b), b, l$  {recall the
                             packing notation}

CPV B,j: Call procedure as variable
          $s \leftarrow s+1$ 
          $C(s) \leftarrow D(b), b, i$  {save TP}
         Go( $C(D(B)+j)$ ) {then transfer}

```

Notice that CP can be regarded as a degenerate case of CPV in which the display need not be restored because the call is within the block where the procedure was declared, guaranteeing that the procedure's context is already in the display.

Also notice that there are two transfer points involved here: the one being created for eventual use by RTN and the one being transferred to.

We pass a procedure name by loading its transfer point. It is called by CPV instead of CP.

The translation of the summation program given above is then

|      |     |      |                               |
|------|-----|------|-------------------------------|
|      | BE  | 0    | begin                         |
|      | JP  | L1   | {get around procedure code}   |
| SUM  | PE  | 0    | integer procedure SUM(F,N); . |
|      | BE  | 2    | begin integer I,S;            |
|      | LC  | 1    | I:=1                          |
|      | STD | 1,0  |                               |
|      | LC  | 0    | S:=0                          |
|      | STD | 1,1  |                               |
| LOOP | LE  | 1    | LOOP:                         |
|      | LV  | 1,0  | if I<N then                   |
|      | LV  | 1,-6 |                               |
|      | LS  |      |                               |
|      | NOT |      |                               |
|      | JIF | L2   |                               |
|      | BE  | 0    | begin                         |
|      | LV  | 1,1  | S                             |
|      | LC  | 0    | {return value place}          |
|      | LV  | 1,0  | I                             |
|      | CPV | 1,-7 | F(I)                          |
|      | ADD |      |                               |
|      | STD | 1,1  | S:=S+F(I)                     |
|      | LV  | 1,0  | I:=I+1                        |
|      | LC  | 1    |                               |

|    |       |      |                                |
|----|-------|------|--------------------------------|
|    | ADD   |      |                                |
|    | STD   | 1,0  |                                |
|    | JP    | LOOP | go to LOOP                     |
|    | EB    |      |                                |
| L2 | LV    | 1,1  | SUM:=S                         |
|    | STD   | 1,-8 |                                |
|    | EB    |      | end                            |
|    | RTN   | 2    |                                |
| SQ | PE    | 0    | integer procedure SQ(X); . . . |
|    | BE    | 0    | begin                          |
|    | LV    | 1,-6 | SQ:=X*X;                       |
|    | LV    | 1,-6 |                                |
|    | MULT  |      |                                |
|    | STD   | 1,-7 |                                |
|    | RTN   | 1    |                                |
|    | EB    |      | end                            |
| L1 | LC    | 0    | {return value for SUM}         |
|    | LL    | SQ   | {procedure label}              |
|    | READ  |      |                                |
|    | CP    | SUM  |                                |
|    | PRINT |      | PRINT(SUM(SQ,READ))            |
|    | EB    |      |                                |
|    | HALT  |      |                                |

### 9.8.8. Implementation of CBN Label Parameters

Algol 60 permits a statement label to be passed to a procedure. Such a label can be used as an error exit, permitting the program to branch back through several levels of procedure calls to some common point. By modern standards of structured programming, this is not a desirable language feature, as it means that no procedure call containing a label can be counted upon returning to the statement following the procedure. However, the language does require this feature, so let us see how a passed label can be implemented.

Here is an example:

```

begin integer procedure RECIP(X,E);
  value X; integer X; label E;
  begin if X=0 then go to E;
    RECIP:=1/X
  end;
  PRINT(RECIP(READ,ER) +
    RECIP(READ,ER))
ER:
end

```

In this program, if either number read is zero the computation of the sum of the reciprocals is cut short by the transfer to ER.

The implementation of label parameters goes much as procedure parameters. We shall use LL and invent a new instruction

JPV B,j: Jump to variable at address B,j  
Go( $C(D(B)+j)$ )

Once again, JP can be viewed as a degenerate case of JPV. Recall that procedure Go reorganizes the display to correspond to the previous environment, that associated with the branch target.

The translation of the above program then is

|       |       |       |                                        |
|-------|-------|-------|----------------------------------------|
|       | BE    | 0     | begin                                  |
|       | JP    | L1    |                                        |
| RECIP | PE    | 0     | integer procedure RECIP . . .          |
|       | BE    | 0     | begin                                  |
|       | LV    | 1,-7  | if X=0 then                            |
|       | LC    | 0     |                                        |
|       | EQ    |       |                                        |
|       | JIF   | L2    |                                        |
|       | JPV   | 1,-6  | go to E                                |
| L2    | LC    | 1     | RECIP:=1/X                             |
|       | LV    | 1,-7  |                                        |
|       | DIV   |       |                                        |
|       | STD   | 1,-8  |                                        |
|       | EB    |       | end                                    |
|       | RTN   | 2     |                                        |
| L1    | LC    | 0     | {return value for RECIP}               |
|       | READ  |       |                                        |
|       | LL    | ER    |                                        |
|       | CP    | RECIP |                                        |
|       | LC    | 0     |                                        |
|       | READ  |       |                                        |
|       | LL    | ER    |                                        |
|       | CP    | RECIP |                                        |
|       | ADD   |       |                                        |
|       | PRINT |       | PRINT(RECIP(READ,ER) + RECIP(READ,ER)) |
| ER    | LE    | 0     |                                        |
|       | EB    |       |                                        |
|       | HALT  |       |                                        |



### 9.8.9. Summary of Procedure Parameter Mechanisms

The possible relationships of actual to formal parameters for Algol-like languages are summarized in figure 9.20. Algol 60 supports only CBV and CBN, while Fortran supports CBR. We have shown all three cases in the figure for the sake of generality.

The actual parameter may be

1. A constant.
2. A variable name or indexed variable name.
3. An expression.
4. A procedure name.

By *expression*, we mean any expression other than a constant, a variable name, or an indexed variable name, which fall under the first two categories. By *procedure name*, we mean that the name of a procedure is passed, to be called within the callee procedure, not a procedure intended to be called before passing the parameter. A passed procedure name will never carry actual parameters when it appears as an actual parameter.

The uses of the formal parameter may be classified as follows:

|                       | ACTUAL PARAMETER |                        |                                |                                  | Procedure name   |
|-----------------------|------------------|------------------------|--------------------------------|----------------------------------|------------------|
|                       | F.P.             | Constant               | Variable                       | Expression                       |                  |
| Variable assigned-to  | CBV              | Affects local copy     |                                |                                  | ERROR            |
|                       | CBR              | ERROR*                 | Affects actual parameter       | ERROR*                           |                  |
|                       | CBN              | ERROR*                 |                                | ERROR*                           |                  |
| Variable fetched only | CBV              | Returns constant value | Variable evaluated before call | Expression evaluated before call | ERROR            |
|                       | CBR              |                        |                                |                                  |                  |
|                       | CBN              |                        | Variable value at fetch        | Expression evaluated at fetch    |                  |
| Procedure call        | (any)            | ERROR                  |                                |                                  | Procedure called |

\*Difficult error to detect in general

**Figure 9.20.** Procedure parameter passing cases, with call-by-value (CBV), call-by-reference (CBR) and call-by-name (CBN).

1. As a variable that is assigned-to somewhere.
2. As a variable that is never assigned-to, but may be fetched.
3. As a procedure name to be called.
4. As an actual parameter to be passed to another procedure.

We shall deal with case 4 separately; in principle, this can always be resolved to one of the other three if the called procedure is considered. Recursive calls that only pass parameters around among each other are barred from consideration.

Now the declarations for a formal parameter in Algol 60 provide the following attributes:

1. Whether CBV or CBN (recall that CBR is not in Algol 60),
2. Whether a procedure or not.

We can assume that the formal parameter attributes are known upon analyzing any call. Note that “assigned-to” is not among the attributes; it can only be determined through an analysis of the procedure statements.

For the sake of generality, we add the attribute REFERENCE to those already in Algol 60, to indicate that the formal parameter is CBR. We can then distinguish CBV, CBN, and CBR from the FP declarations.

Now consider passed procedure names (figure 9.20). A PROCEDURE formal parameter can only be associated with a procedure name actual parameter; any other actual parameter is a compile-time error and is easy to detect. Also, any use of a PROCEDURE formal parameter other than as a procedure call is illegal.

Next consider a constant actual parameter. A constant should not be assigned-to, but may be fetched. In principle, the compiler can detect such errors, but an elaborate mechanism is needed, as we shall see. For CBV, “assigning-to” a constant merely means that the local copy of the constant is altered; the actual parameter is unaffected. Such a use of a CBV formal parameter is valuable, as a means of conserving local variable space. For example, procedures MAS, SETP and MMAS (above) use this characteristic.

An expression’s actual parameter also should not be assigned-to, but can be fetched. Here, CBR and CBN are fundamentally different. In CBR, the expression is evaluated just before the call, at the moment at which the actual parameter is processed by the compiler. In CBN, the expression is not evaluated before the call, but is evaluated afresh upon each appearance of its corresponding formal parameter within the called procedure. Clearly, a thunk must be created by the compiler upon processing the actual parameter for CBN. The thunk label is passed as the actual parameter, and the thunk is called when the formal parameter reference is processed in the procedure. The thunk will then evaluate the expression and return its value, as needed.

The CBN and CBR expression evaluation result can clearly be different. For example, if the expression contains free variables, their value may be changed by the callee between formal parameter references. Then CBN will return different results on different references, whereas CBR will always return the same result.

Of course, CBR can return different results on different references, but only because a specific assignment to that formal parameter was made, or another CBR formal parameter carries the same address and was assigned-to. The latter situation is called *aliasing*—two apparently different formal parameters refer to the same data area. Aliasing is a fertile source of program errors.

If the actual parameter is a simple gr indexed variable, then an assignment to it within the callee procedure makes sense and is a useful operation. However, again, CBR and CBN can yield different results, especially for an indexed variable. The index is an expression in general, and in CBN the index is evaluated on each reference. In CBR the index is evaluated before the call, and the appropriate indexed variable address is passed; the index is thereupon effectively fixed while the callee is active.

The following (nonsensical) program illustrates all the cases of figure 9.20.

```
begin
  integer I,J,K;
  integer array IA[0:15];
  procedure Q(A,B,C,D);
    value A;      {A is CBV}
    reference B;   {B is CBR}
    integer A,B,C; {C is CBN}
    procedure D; {D is a procedure name}
  begin
    {variables assigned-to}

    A:=10;      {OK in all cases}
    B:=10;      {OK except for
                 constant or expression AP}
    C:=10;      {OK except for
                 constant or expression AP}
    D:=10;      {ERROR}

    {variables fetched}

    I:=A;       {OK in all cases}
    I:=B;       {OK in all cases}
    I:=C;       {OK in all cases}
    I:=D;       {ERROR}
```

```

{variables called}

A;      {ERROR}
B;      {ERROR}
C;      {ERROR}
D(I,I,I,Q) {OK, assuming the parameters
           match the called procedure}

end;

{now we give some typical calls}

Q(5,5,5,5); {constants passed—3 errors}
    ** ↑ {*=hard to detect, ↑=easy to detect}

Q(I,I,I,I); {simple variables—one error}
    ↑

Q(IA[I+J], IA[I+J], IA[I+J], IA[I+J]);
                                ↑ {1 error}

Q(I+J, I+J, I+J, I+J); {3 errors}
    *      *      ↑

Q(Q,Q,Q,Q); {procedure names—3 errors}
    ↑ ↑ ↑
end

```

The errors marked \* in figure 9.20 and in the above program are difficult to detect in a one-pass compiler, but possible to detect in a two or more pass compiler. These depend on detecting an assigned-to condition of a CBN or CBR formal parameter, when the actual parameter is a constant or expression. In order to detect such an error upon processing a call, the attribute “assigned-to” or “not assigned-to” must be associated with each CBN and CBR formal parameter. The “assigned-to” attribute is set if either of the following conditions is met:

1. The formal parameter is set somewhere within the procedure, i.e., as the left member of an assignment, the target of a READ, etc.
2. The formal parameter is passed by reference or name to another procedure, such that the corresponding formal parameter is “assigned-to.”

The first condition requires one pass to establish “assigned-to” for its formal parameters. More passes are needed to check out the second condition if the first is not met. For example, the following program requires three passes to fully establish the “assigned-to” status for each of the formal parameters:

```

begin
  integer I;
  procedure Q1(A1);
    integer A1; {A1: CBN}
    begin
      Q2(A1);
    end;

  procedure Q2(A2);
    integer A2;
    begin
      Q3(A2);
    end;

  procedure Q3(A3);
    integer A3;
    begin
      A3 := 15
    end;

    Q1(14);
    Q1(I + J); {each of these calls
               contains an error}
end

```

Note that the FP is CBN in each procedure. On the first pass, A3 only is “assigned-to.” On the second pass, because A2 is passed to Q3 by CBN, A2 is “assigned-to.” On the third pass, A1 is found to be “assigned-to.” Obviously, programs can be devised that will require an arbitrary number of passes to fix the “assigned-to” status of all FP’s.

A more practical strategy is to not attempt a compile-time check of these errors, in the hope that they will be manifest at run-time. This is not in the best interests of the user, but certainly simplifies the compiler.

#### 9.8.10. Call by Name Implementation

Figure 9.20 illustrates a special problem with CBN—an efficient evaluation approach seems to depend on the nature of the actual parameter. If the AP is a constant, it may simply be loaded on the stack and fetched by a LD. If it is a simple variable, the variable’s address may be placed on the stack and evaluated by the pair of instructions LA, CONT. However, an indexed variable and an expression require a thunk—they are to be evaluated at the point of fetch (or store, in the case of an indexed variable).

Now we cannot afford to provide three different mechanisms within a procedure to deal with the three different AP cases. We need one common

method to deal with all three in a uniform manner, and that has to be a thunk in every case.

If a thunk is needed for an AP, the procedure caller will construct it—the caller after all knows the character of the AP. In every case, the thunk should return the address of something, since an address is needed for a variable that is assigned-to. Then the callee need only call the thunk associated with the FP and use the returned address in a standard way to either fetch the value or store into a variable.

The thunk for a simple variable should return the address of the variable. The thunk for an indexed variable should evaluate the index and return the address of the indexed variable. The thunk for a constant should place the constant in some addressable location (we discuss this later), and return the address of that location. Finally, the thunk for an expression should evaluate the expression, store the result in an addressable location, and return the address of that location.

Now what location is appropriate for a constant or expression temporary value in a thunk? If a temporary location is allocated by the thunk during its execution, its address could be returned; however, the address will point to a location above the stack top upon return, an illegal address in general. Clearly, a temporary is needed just before the thunk call. We therefore must adopt the thunk calling convention:

```
LC 0    {word to receive the returned address}
LC 0    {word to receive a temporary value, if any}
CPV . . . {thunk call}
```

Obviously, the two LC's can be combined into an INCS 2. Then if the address is wanted from the thunk, the instruction

```
DECS 1
```

following the thunk call is appropriate.

If the value is wanted, we need a special instruction RAV as follows:

```
RAV: Replace address by value
       $C(s-1) \leftarrow C(C(s-1))$ 
       $s \leftarrow s-1$ 
```

RAV uses the returned address in location  $s-1$  to fetch the value, replaces the address by the value, then drops the stack top. This peculiar instruction should only be used after a thunk call to obtain a value.

Now some typical thunks might be written as follows, assuming the thunk is written at block level 3 (the block level it is written in is immaterial, except that during execution, its environment must be that of the caller setting it up):

```

{think for a constant c}
PE      3
BE      0      {no local variables, but b=4 now}
LC      c      {load the constant}
STD     4,-6   {value is just below stack marker}
LA      4,-6
STD     4,-7   {address is below value field}
EB
RTN     0

```

```

{think for an expression A*B}
PE      3
BE      0
LV      A      {whatever A's textual address is}
LV      B
MULT
STD     4,-6
LA      4,-6
STD     4,-7
EB
RTN     0

```

```

{think for an indexed variable X[I+J]}
PE      3
BE      0
LA      X
LV      I
LV      J
ADD
ADD
STD     4,-7
LV      4,-7
CONT
STD     4,-6
EB
RTN     0

```

Since some of these instructions will always appear in a think definition, they might be combined into single instructions, for the sake of efficiency. For example, the combination

```

STD     4,-6
LA      4,-6
STD     4,-7

```

needs only the thunk block level (4). Also, the entry pair PE, BE and the exit pair EB, RDN are standard for a thunk. Special thunk call instructions that return either address or value could also be devised that combine some functions of the above call sequences.

### Example

The following example illustrates CBN and CBV for several kinds of actual parameter and generation of the necessary thunks for CBN. Since CBR is an obvious variation on CBV, we will not illustrate CBR.

```
begin integer A,B;
  integer procedure INC(X,CHANGE);
    value CHANGE; integer X;
    boolean CHANGE;
  begin INC:=X+1;
    if CHANGE then X:=X+1
  end;
  A:=INC(0,FALSE); {constant by name}
  B:=INC(A,TRUE);  {variable by name}
  B:=INC(A*B,FALSE); {expression by name}
  begin integer procedure INCA;
    begin INCA:=INC(A,TRUE) end;
    A:=INC(INCA,FALSE)
      {procedure name passed}
  end
end
```

The textual addresses of A,B,X and CHANGE are:

```
A: [0,0]
B: [0,1]
X: [1,-7]
CHANGE: [1,-6]
```

The TA of the return value of INC will then be [1,-8].

The translation of the previous program is given next.

|     |      |    |                                        |
|-----|------|----|----------------------------------------|
|     | BE   | 2  | begin integer A,B;                     |
|     | JP   | L1 | {branch around procedure code}         |
| INC | PE   | 0  | integer procedure INC(X,CHANGE); . . . |
|     | BE   | 0  |                                        |
|     | INCS | 2  | {locations for thunk address, value}   |



```

CPV      1,-7 {call thunk for X}
DECS     1      {leave address behind}
LC        1
ADD
STD      1,-8 INC:=X+1
LV        1,-6
JIF      L2    if CHANGE then
INCS     2      {thunk call for X again}
CPV      1,-7
DECS     1
INCS     2
CPV      1,-7 {another thunk call for X}
RAV      {value wanted this time}
LC        1
ADD
ST        X:=X+1
L2      EB      end
RTN      2
L1      LC      0 {return value cell}
LL       TH1
JP       L3
TH1     PE      0 {start of thunk for constant 0}
BE       0
LC       0      {the constant}
STD      1,-6
LA       1,-6
STD      1,-7
EB
RTN      0
L3      LC      FALSE
CP       INC
STD      0,0    A:=INC(0,FALSE)
LC       0
LL       TH2
JP       L5
TH2     PE      0 {start of thunk for variable A}
BE       0
LA       0,0
STD      1,-7
LV       1,-7
CONT
STD      1,-7 {value may or may not be wanted}
EB
RTN      0

```

```

L5    LC    TRUE
      CP    INC
      STD   0,1  B:=INC(A,TRUE)
      LC    0
      LL    TH3
      JP    L8
TH3   PE    0      {start of thunk for A*B}
      BE    0
      LV    0,0
      LV    0,1
      MULT
      STD   1,-6
      LA    1,-6
      STD   1,-7
      EB
      RTN    0
L8    LC    FALSE
      CP    INC
      STD   0,1  B:=INC(A*B,FALSE)
      BE    0      begin
      JP    L10
INCA  PE    1      integer procedure INCA;
      BE    0
      LC    0
      LL    TH2 {label of thunk for A}
      LC    TRUE
      CP    INC
      STD   2,-7 INCA:=INC(A,TRUE)
      EB          end
      RTN    0
L10   LC    0
      LL    TH4
      JP    L11
TH4   PE    1      {start of thunk for INCA}
      BE    0
      LC    0
      CP    INCA
      STD   2,-7
      LA    2,-6
      STD   2,-7
      EB
      RTN    0      {end of INCA thunk}
L11   LC    FALSE
      CP    INC

```

```

STD    0,0  A:=INC(INCA,FALSE)
EB      end
EB      end
HALT

```

### 9.8.11. CBV versus CBN

In the previous section, we contended that the callee (the called procedure) should be ignorant of the nature of the AP's in any particular call. By the same token, it seems desirable that the caller should also be ignorant of whether its parameters are passed by value or by name. If the caller does know through a required declaration of the procedure, well and good, but Algol 60 doesn't require such a local declaration.

In order to achieve this kind of complete isolation of caller and callee—neither knowing the conventions of the other—the caller must assume the worst and always pass a thunk label. The callee must always expect a thunk label. This label is obviously inefficient in stack space and execution time, but is the price that must be paid for such isolation.

Because this general situation is so clumsy in implementation, most modern languages do not support CBN.

Either CBR or CBV may be dropped to support quite general procedure calls. In Fortran, only CBR is supported. If only CBV is supported, a pointer may be passed by value, effectively supporting CBR.

## 9.9. Summary of AOC Instructions

### 1. Stack manipulation instructions

```

LC    c:  Load constant
         $s \leftarrow s + 1$ 
         $C(s) \leftarrow c$ 

LA    B,j: Load address
         $s \leftarrow s + 1$ 
         $C(s) \leftarrow D(B) + j$ 

LV    B,j: Load value
         $s \leftarrow s + 1$ 
         $C(s) \leftarrow C(D(B) + j)$ 

LL    l:  Load label
         $s \leftarrow s + 1$ 
         $C(s) \leftarrow D(b), b, l$ 

```

STD B<sub>j</sub>: Store direct  
 $C(D(B)+j) \leftarrow C(s)$   
 $s \leftarrow s-1$

ST: Store  
 $C(C(s-1)) \leftarrow C(s)$   
 $s \leftarrow s-2$

RAV: Replace address by value  
 $C(s-1) \leftarrow C(C(s-1))$   
 $s \leftarrow s-1$

CONT: Contents  
 $C(s) \leftarrow C(C(s))$

INCS: Increment stack pointer ( $C(s) \geq 0$ )  
 $C(s+C(s)) \leftarrow s$   
 $s \leftarrow s+C(s)$

INCS n: Increment stack pointer ( $n \geq 0$ )  
 $s \leftarrow s+n+1$

DECS: Decrement stack ( $C(s) \geq 0$ )  
 $s \leftarrow s - C(s) - 1$

DECS n: Decrement stack ( $n \geq 0$ )  
 $s \leftarrow s-n$

## 2. Expression computing instructions

ADD:  $C(s-1) \leftarrow C(s-1) + C(s)$   
 $s \leftarrow s-1$

SUB:  $C(s-1) \leftarrow C(s-1) - C(s)$   
 $s \leftarrow s-1$

MULT:  $C(s-1) \leftarrow C(s-1) * C(s)$   
 $s \leftarrow s-1$

DIV:  $C(s-1) \leftarrow C(s-1) / C(s)$   
 $s \leftarrow s-1$

NEG:  $C(s) \leftarrow -C(s)$

EQ:  $C(s-1) \leftarrow$  if  $C(s-1) = C(s)$  then  
 TRUE else FALSE  
 $s \leftarrow s-1$

LS:  $C(s-1) \leftarrow$  if  $C(s-1) < C(s)$  then  
 TRUE else FALSE  
 $s \leftarrow s-1$

GT:  $C(s-1) \leftarrow$  if  $C(s-1) > C(s)$  then  
           TRUE else FALSE  
 $s \leftarrow s-1$

NOT:  $C(s-1) \leftarrow$  if  $C(s) = \text{FALSE}$  then  
           TRUE else FALSE

AND:  $C(s-1) \leftarrow$  if  $C(s-1)=\text{TRUE}$  and  
                            $C(s)=\text{FALSE}$  then  
                           TRUE else FALSE  
 $s \leftarrow s-1$

OR:  $C(s-1) \leftarrow$  if  $C(s-1)=\text{TRUE}$  or  
                            $C(s)=\text{FALSE}$  then  
                           TRUE else FALSE  
 $s \leftarrow s-1$

### 3. Block instructions

BE n: Block entry  
        $C(s+1) \leftarrow s+n+2$        {save WP}  
        $C(s+2) \leftarrow D(b)$        {save SP}  
        $b \leftarrow b+1$   
        $D(b) \leftarrow s+3$   
        $s \leftarrow s+n+2$

EB: Exit block  
        $s \leftarrow D(b)-3$   
        $b \leftarrow b-1$

### 4. Jump instructions

JP l: Jump  
        $i \leftarrow l$

JIF l: Jump if false  
       if  $C(s)=\text{FALSE}$  then  $i \leftarrow l$   
        $s \leftarrow s-1$

JPV B,j: Jump through variable  
            $\text{Go}(C(D(B)+j))$   
           {See section 9.6.4}

LE B: Label entry  
        $b \leftarrow B$   
        $s \leftarrow C(D(b)-2)$

### 5. Procedure instructions

CP 1: Call procedure  
 $s \leftarrow s + 1$   
 $C(s) \leftarrow D(b), b, i$   
 $i \leftarrow 1$

PE B: Procedure entry  
 $b \leftarrow B$

RTN m: Return  
 $Go(C(s))$  {see section 9.6.4}  
 $s \leftarrow s - m - sm$

CPV B,j: Call procedure through variable  
 $s \leftarrow s + 1$   
 $C(s) \leftarrow D(b), b, i$   
 $Go(C(D(B) + j))$

#### 6. Input-output

READ:  $s \leftarrow s + 1$   
 $C(s) \leftarrow$  next item on input; advance read head

PRINT: Print  $C(s)$   
 $s \leftarrow s - 1$

HALT: Stop the machine

#### 7. Array instructions

MAS B,j,n: Make array space  
 {see section 9.7.1}

AVA: Array variable address  
 {see sections 9.7.1, 9.7.2}

MMAS B,j,n: Make matrix array space  
 {see section 9.7.2}

### 9.10. Bibliographical notes

Much of the material in this chapter is adapted from Randell and Russell (Randell [1964]), who first defined a set of instructions for implementation of Algol 60. Ingerman [1961] gives an algorithm for the rearrangement of an OWN array that is redimensioned in place. Sattley [1961] discusses space allocation and dope vectors for Algol 60 local variables and arrays.

Bauer [1968] describes an Algol 60 implementation in some detail. Berkeley [1964] describes Lisp implementation, of interest for the automatic

allocation of list elements and garbage collection of dead elements. Griffiths [1974a] and Hill [1974] contain a comprehensive review of run-time storage management, with special attention to the management of Algol 68 data structures.

Lee [1967] discusses some special addressing problems that arise in Fortran compilers, in particular, allocation considering equivalences. McKeeman [1970] contains complete source for the XPL compiler, a derivative of Algol 60 designed for compiler and systems programming.

Concurrent garbage collection was first proposed by Steele [1975], and later refined by Dijkstra [1976a] and Gries [1977].

Wirth [1971a] describes a Pascal implementation on a CDC 6000 system. His implementation could be adapted to any computer system, however. Wirth [1976c] also gives a complete compiler for a small language, written in Pascal.

# OBJECT CODE AND MACHINE ARCHITECTURES

---

### 10.1. Introduction

The object code of a compiler can take a variety of different forms: (1) another high-level language, (2) a macro language intended for a macro assembler, (3) a symbolic assembly language, (4) a special intermediate language (IL), (5) relocatable loader tables, with symbolic code references, (6) absolute loader tables, or (7) absolute machine code.

The object code sections of a compiler are, as a rule, more difficult and complicated to design for the latter cases than for the former.

We will not discuss cases (1), (2), or (3) in this chapter. Such a compiler is essentially a string processor and may often be implemented directly through a SDTS as described in chapter 7. Much depends on the ability of the object language to express the concepts of the source language. It may be necessary to develop special algorithms in the compiler in order to get around any limitations in the object language. For example, if the object language is symbolic assembly language and the source is a block-structured language, name conversions will be needed, since most assemblers support only one block level.

Some compilers generate a special intermediate language (IL) that can be further translated to object code on two or more different machines. The intermediate language is usually a simple translation of the abstract syntax trees generated by the compiler. Certain optimizations can be performed directly on an intermediate language, however, in general IL optimization is difficult.

Most commercial compilers generate linking relocating loader tables.

*Linking* refers to a capability of connecting procedure calls with their procedures, or variable references with their external declarations, after compilation. This capability permits a user to compile only a few procedures at a time and collect them all together later. The advantage of linking after compiling is that a large program may be brought together in manageable pieces, and errors may be repaired with a minimum of compilation. Without a linking loader, the entire program must be compiled after even the slightest source change.



*Relocation* refers to a capability of locating a program at an arbitrary position in physical memory just before execution. Clearly, linking requires relocation, since the procedures are generally assigned contiguous portions of memory after compilation is complete. Relocation is essential in a multiprocessing system, so that several independent programs can be executed in a time-shared manner without necessarily unloading and reloading memory for each program. The distinct programs can be assigned to different regions of memory, and the processor can then be rapidly switched from one program to another.

In this chapter, we shall review some intermediate code structures, review three machine architectures and their loader tables, and then close with a description of a generalized code generator system. Some special optimizations for code generation, register allocation, and code improvement are discussed in chapter 11.

## 10.2. Intermediate Languages

An intermediate language, or IL, usually consists of a relatively small number of simple operations, combined to form a list semantically equivalent to the source program. The operations are chosen to express the source language needs completely, with as little redundancy as possible, and are organized in a simple uniform manner. An IL is not intended to be written or read by a person, and therefore needs no syntactic window-dressing.

Some common IL's are

- Quadruples (or quads).
- Triples.
- Postfix.
- Prefix.

A *quad* expresses a single unary or binary operation as follows:

$\langle \text{operation} \rangle, \langle \text{operand1} \rangle, \langle \text{operand2} \rangle, \langle \text{result} \rangle$

where the  $\langle \text{operation} \rangle$  is applied to the two operands to yield the  $\langle \text{result} \rangle$ . The result and the operands are references to some attribute data structure and may be declared program variables or temporary variables. Some other section of the compiler is responsible for assigning physical memory to the result and operands; indeed, some temporaries may be assigned to registers and not to physical memory. The IL is then a sequential list of quads.

For example, the expression  $A*B + C*D$  can be represented as the quad sequence

```

*, A, B, T1      {T1 is a temporary result}
*, C, D, T2      {T2 is another temporary}
+, T1, T2, T3    {expression result is T3}

```

Note that we assign temporary cell names as needed; a separate algorithm can determine the number of temporaries actually needed and assign registers and memory.

A *triple* is essentially a quad, except that its result is referred to by the location of the triple, rather than explicitly. Every triple is assumed to return some result, and its result can be referred to in some later triple. The advantage of triple notation is that explicit temporary names are unnecessary. In triple notation, the expression  $A*B + C*D$  can be expressed as

```

(1)  *, A, B
(2)  *, C, D
(3)  +, (1), (2)

```

Here, the (1) in the third triple refers to the result generated by the first triple. Of course, a literal “1” must be distinguishable from a triple number.

Some examples of operations that can be expressed as a list of quads or triples are

- Arithmetic operations.
- Math functions.
- Assignments.
- Indexing.
- Comparisons (result is a truth value).
- Logical operators.
- Branches, conditional or unconditional.
- Conversion of real to integer, or vice versa.
- Begin or end a block.
- Procedure or function call.
- Allocation of array storage.

Declarations and control structures may also be translated to quads or triples. A declaration may require space allocation and the fixing of an address label on the stack. Control structures are usually broken into a sequence of statements separated by branches. A case statement requires a few special quads to express a branching table.

Indexing for array or structure access can be handled by indexed “load” or “assign” quads:

XL, A, I, T     {indexed load; equivalent to  $T := A[I]$ }  
 XS, T, A, I     {indexed store:  $A[I] := T$ }

Then the statement  $C := A[i, B[j]]$  can be translated to the quads

\*, i, d1, T1     {d1 is the first dimension for A}  
 XL, B, j, T2     { $T2 := B[j]$ }  
 +, T1, T2, T3 {T3 is final index for A}  
 XL, A, T3, T4 { $T4 = A[i, B[j]]$ }  
 :=, T4, , C     {assignment is a unary operation}

Quads or triples can easily be generated from a binary tree. The tree is scanned in pre-order, and the internal nodes are numbered in the usual left-to-right evaluation order. Each internal node  $N$  then translates to a triple whose number is the node number, whose operation is the operator associated with node  $N$ , and whose operands are node  $N$ 's children. Thus the tree of figure 10.1, for the expression  $A*B + C - D/E$  translates to the triple list

- (1) \*, A, B
- (2) +, (1), C
- (3) /, D, E
- (4) -, (2), (3)

A source program can clearly be expressed as a list of quads or triples. Some reductions on a list are possible without a knowledge of the target machine architecture. For example, constant arithmetic may be subsumed by noticing that certain quads perform constant arithmetic; their result may be computed at compile time and substituted into those quads that require the result. We shall discuss such optimizations in chapter 11.

The flow of control through a program can also be easily analyzed through quads or triples; we need only scan the list, looking for branches. The control flow can be traced without regard to the data values of the variables, provided that the conditional branches are to a finite number of known, specified locations, and also provided that no quad can be modified by a run-time operation.

Some machine-dependent optimizations are also possible on quads, e.g., the so-called *peep-hole optimizations*. A *peep-hole optimization* examines a short sequence of contiguous quads in an attempt to recognize certain patterns that can be combined into a single quad with a special operation. For example:

1. {+, I, 1, T1}, {:=, T1, , I} stems from  $I := I + 1$  and might be coded INCM I.

2. A STOR I from some register R, followed by a LOAD I, is fairly common; the LOAD I can be replaced by no operation or by a register-to-register copy.
3. An indexed reference with a constant index can sometimes be replaced by a simple direct reference with an appropriate address.

### Postfix and Prefix

The use of postfix or prefix notation is convenient as a linear representation of a tree structure. These are generated by a left-to-right natural order scan of an abstract syntax tree. The leaf operands are written as they are encountered. A prefix expression is obtained by emitting an operator (associated with an internal node) upon first encountering it; a postfix expression is obtained by emitting an operator upon last encountering it. Thus the tree of figure 10.1 can be expressed as

- + \* A B C / D E

in prefix and as

A B \* C + D E / -

in postfix.

No parentheses are required, unless an operator takes an indefinite number of parameters; even then, the first operand of such an operator could be the number of parameters expected.

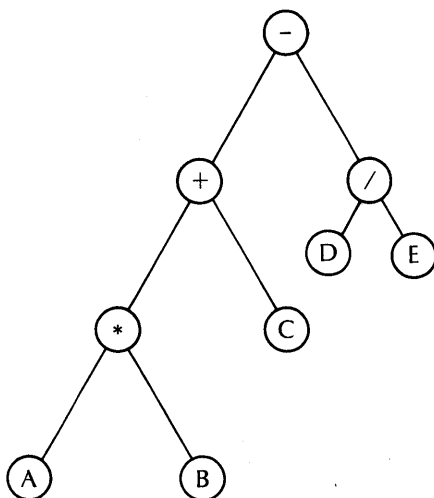


Figure 10.1. An expression tree.

A postfix or prefix string is not convenient for optimization purposes; it must first be transformed into a quad or triple list or into a tree structure.

### Exercises

1. Write the following Fortran statements as quads. Use the dimension statements if necessary. Note that in Fortran, the least index corresponding to some dimension is 1.

```
DIMENSION X(16,25),Y(55);
X(3,7) = (I+J+X(I,J)) * Y(5)
I = I*(J+Y(I+5))
```

2. Design triples to support indexed loads and stores, then write the statements of exercise 1 in triple notation.
3. Design triples to support a procedure call with parameters. One triple may be followed (or preceded) by a list of triples that specify the passed parameter values. What can be done about the return value of a procedure? How can call by value and call by reference be distinguished?
4. Sketch an algorithm that accepts a list of triples and then subsumes constant arithmetic and identity operations (e.g.,  $\{+, 25, 17\} = 42$ , or  $\{*, 0, (7)\} = 0$ ) wherever possible under commutativity but not under associativity or distributivity. It should essentially scan the triple list in execution order, evaluate those triples that represent constants or identities, then substitute the results in later triples that require them, evaluating them if possible, etc.
5. Sketch an algorithm that identifies useless triple operations. A useless triple can be removed from the sequence without affecting the semantic validity of the program.

## 10.3. The Pros and Cons of Intermediate Languages

An IL may be used in a compiler to facilitate optimization or compiler transportability. The former objective will be discussed at length in the next chapter. We shall see that the most convenient intermediate form is a tree or directed acyclic graph. Let us therefore discuss the latter purpose.

Suppose that it were possible to design an IL that could support m

different languages (e.g., Fortran, Basic, Algol). Further suppose that we wish to implement each of these languages on  $n$  different target machines (e.g., IBM 360, HP 2100, DEC 10). Without an IL,  $mn$  compilers would be required, one for each language and each machine. Some of the software could be shared among the compilers for a particular machine, but in general each of these would be a separate project. Given an IL, on the other hand, only  $m + n$  projects would be required— $m$  projects to implement translators from each of the source languages to IL and  $n$  projects to implement code generators from the IL to the various target machines. Each compiler then consists of a “front end” that translates source to IL, coupled to a “back end” that translates IL to object code. The  $mn$  compilers are constructed (in principle at least) from the  $m + n$  components.

Unfortunately, languages and machines are of such diversity that such an undertaking is likely to fail. From the language standpoint, two apparently similar languages are often quite different in detail and require special IL structures to support them. Let us examine just three languages, Basic, Fortran and Algol, as possible candidates for a common IL:

1. The Basic data types are simple numerics, arrays, and strings. The numeric type is usually represented as a floating-point number and is printed as an integer if its fractional part is zero. The Basic strings carry a string count or end of string indicator. Fortran supports several different data types, but not string. Algol supports reals and integers; some Algol implementations also support characters and character arrays. Clearly, an IL must contain memory allocation indicators for each of these types, some of which are used in only one language.
2. The Algol block structuring and procedure call mechanisms are not fully used in Fortran or Basic. Three parameter passing mechanisms must be supported: call by value (Algol), call by reference (Basic and Fortran), and call by name (Algol). These must be matrixed with the data types in a rather complicated way—Basic strings are not passed by value or by name, and Basic strings are not passed in the same way as Algol character strings. The Algol system is the most general, but would be inefficient for Fortran or Basic. Hence the back end should know whether a full Algol system is required or not.
3. Consider the FOR statement (Algol). Comparable structures exist in Basic and Fortran, but have different definitions. For example, the Fortran DO statement is always executed once, regardless of the parameter value viz-a-viz the limit, while the Algol FOR is tested before a first execution. The Algol step size may also be negative, while the Fortran step size can only be positive.
4. Fortran provides a number of special features not found in either of the other two languages: e.g., COMMON, statement functions, equiva-